PROJECT 2 · TASK 1—4 · 按关键步骤学习

从“调用程序”到“管理任务”

关键代码整段展示
逐句理解每一次启动、暂停与恢复

目标 → 完整实现块 → 行号解释 → 状态推演 → 验证现象

20个步骤 · 49段原代码 · 844个代码行次 · 464条解释

依据你的Project1-current与当前Project2-Acore源码包
2026年11月12日 · 原代码不改 · 彩色可检索PDF

围绕实验目标挑选代码，不把每个文本差异都当重点。完整宏与函数保留，重复寄存器指令按规则解释。

第一次启动 · PCB与队列 · switch_to · 互斥阻塞 · ecall/sret · sleep · timer/idle

阅读路线与目录

先看目标、状态图与两种栈帧

1. 阅读起点：为什么普通函数调用不够用了
2. Task 1 · 数据模型：先记住任务，再记住两种现场
3. Task 1 · 就绪队列：用双向链表组织“谁可以运行”
4. Task 1 · 创建任务：加载程序只是第一步，还要分栈并建PCB
5. Task 1 / 3 · 第一次启动：伪造一份“以后恢复时会用到”的现场
6. Task 1 · 选择下一个任务：FIFO轮转：该回队尾的才回队尾
7. Task 1 · 真正切换CPU：switch_to：为什么一个ret能回到另一个任务
8. Task 1 · 主动让出CPU：接好跳转表，让yield到达调度器
9. Task 2 · 竞争与临界区：原子交换、关中断与等待队列各做什么
10. Task 2 · 共享同一把锁：同key返回同槽，索引必须有效
11. Task 2 · 阻塞与移交：拿不到锁就睡，释放时把所有权交给等待者
12. Task 3 · 用户发起请求：把服务号和参数放入寄存器，再执行ecall
13. Task 3 · 异常进入内核：先找可靠的PCB，再选栈和保存现场
14. Task 3 · 分发系统服务：根据a7查表，结果放回a0，ecall的PC加4
15. Task 3 · 返回用户任务：恢复完整帧，最后用sret而不是ret
16. Task 3 · 睡眠与唤醒：睡眠是进入等待队列，不是占着CPU空转
17. Task 4 · 时钟抢占：任务不主动yield，内核也能收回CPU
18. Task 1—4 · 启动顺序与idle：最后接总线：准备完成后才让任务运行
19. 构建与装载前提 · 聚焦关键变化：哪些任务入镜像、入口放哪里、装完才可取指
20. 服务边界 · 输出与错误返回：字符串输出为什么还要检查指针

来源、选择范围与许可证

最后：现象、对应代码、本次运行结果与lab命令

先看目标，再沿执行链读代码

这份讲义按当前Project2-Acore的Task1—4组织：主动切换、互斥阻塞、U态系统调用与睡眠、时钟抢占。依据是仓库README、完成版源码及课程框架差异文件；本包没有实现Task5，不能把保留的fly1—fly5文件当成已完成接口。

你要求的是“如何完成实验的关键步骤”，所以这里不按diff里每一个加减号逐行扫。格式调整、版权头、一般输出库和保留原样的P1链接脚本不重复深讲；任务栈、队列、异常与锁等关键依赖，即使部分声明来自框架，也完整展示并说明来源。

全书选出20个步骤、49段连续原代码，共844个代码行次、464条解释。每个关键函数或宏先整段列出，再按原行号解释；所选代码没有插入省略号。重复寄存器搬运按相同规则分组，原始每条指令仍保留。

原文直接从两个压缩包读取，保留文件路径和行号。代码左侧“+”表示该行相对P1新增或替换；未标记行是保留的必要上下文。这不代表所有“+”都由本次编写：文件来源标签还区分课程框架与后续实现。

任务	要形成的闭环	重点步骤
Task1	PCB / 独立栈 → 初始现场 → 就绪队列 → 调度 → 保存/恢复 → 主动yield	02—08
Task2	同key同锁 → 原子竞争 → 失败阻塞 → 释放移交 → 唤醒后继续	09—11
Task3	U态ecall → 选内核栈/保存 → 服务分发 → 恢复/sret；sleep到期回就绪	12—16
Task4	注册timer → 设置下一次时刻 → 抢占调度 → idle也能唤醒	17—18
共同前提	构建任务与地址对应、装载可取指、用户输出参数检查	19—20

读每一段时，先回答四个问题

1. 输入是什么：参数、当前PCB、队列和寄存器在进入前是什么状态？
2. 改变什么：写了哪个字段、哪个寄存器、哪几条链表连接？
3. 下一步由谁执行：普通return、换到另一任务、异常返回，还是继续等待？
4. 哪项现象能验证它：双计数、持锁竞争、非法调用返回、睡眠下界还是异步抢占？

任务状态图

READY：在就绪队列

调度 →

← yield

RUNNING：当前执行

阻塞 →

BLOCKED：在等待队列

唤醒：BLOCKED → READY；不是立刻变 RUNNING

同一个 PCB 在状态与队列间迁移；一个 list 节点不能同时挂到两个队列。

两种帧先分开记

高地址：ktop（内核栈顶）

regs_context_t：288 字节

32 个 GPR 槽 + sstatus / sepc / stval / scause

保存的 sp = utop；sepc = entry；SPP 决定模式

regs = ktop − 288

switchto_context_t：112 字节

ra = ret_from_exception；保存的 sp = sw

sw = regs − 112

task->kernel_sp 初始指向 sw；栈继续向低地址增长。

另有一页任务栈：PCB 记录其基址，utop 为高地址栈顶；与这页内核栈分开。

对象	含义	典型写入者
task_info_t（48字节）	程序在镜像里的名字/偏移/大小/入口；不是运行中的任务	宿主createimage
pcb_t	任务身份、状态、栈位置、队列节点、唤醒时刻	初始化与调度/锁/sleep
switchto_context_t（112字节）	C调用边界的ra/sp/s0—s11	switch_to或首次预制
regs_context_t（288字节）	被打断时全部GPR槽与返回相关CSR	SAVE_CONTEXT或首次预制

代码读法

x->field表示访问指针x所指对象的成员；&x取地址；*p访问p指向的对象。函数指针表中存的是入口地址，填表不执行服务。ret取ra为PC，sret取sepc并恢复特权级相关状态；sp决定从哪块栈读回保存的数据。

反复出现的寄存器，先查这张表

名称	本书中的用途
ra / sp / tp	普通返回地址 / 栈顶 / 内核current_running绑定的PCB指针；用户tp要原样保存
a0—a4 / a7	系统调用参数 / 系统调用号；返回值用a0
CSR	控制与状态寄存器的统称；不是普通的C内存变量
stvec / sscratch	S态异常入口 / 正常边界上用于定位当前PCB的锚点
sepc	异常返回PC；ecall处理后加4，时钟中断不跳过用户指令
sstatus.SPP	记录陷入前模式，也决定sret回U态还是S态
sstatus.SIE / SPIE	S态全局中断允许 / 异常返回时恢复SIE所用的保存位
sie.STIE	单独允许S态定时中断源；与全局SIE不是同一寄存器中的同一位
rdtime / time_base	硬件tick读数 / 每秒tick数；两者配合才可换算秒数

三层接口要分开：BIOS入口提供固件服务；固定地址jmptab是普通函数指针表；syscall[]是内核在ecall异常之后使用的服务分发表。号码相似或都用a7，并不表示它们属于同一层。

> **本书的验证范围**
>
> 本次重新执行了镜像10项检查、四阶段通用QEMU检查、默认六任务组合和单probe的idle唤醒。运行器在临时副本中明确重定位地址并使用独立测试BIOS；原课程源码不变。课程QEMU/U-Boot、实际块设备、loadbootd/DASIC和Pynq板端仍未实测。

Project2-Acore压缩包 SHA-256：`2ab7415ff127705c660a5ed35255e53beab81bf185b5ddcde4c9556a5cab9a20`

阅读起点

01 为什么普通函数调用不够用了

这一步要完成什么

把P1“加载后调用”的执行模型，与P2“准备多个任务并调度”的目标分开。后续每一步都为保存、恢复、选择任务服务。

与P1／前一步的关系：P1已经能按名字找到任务、从镜像读出代码并取得入口。P2继续利用这个加载能力，新增的是任务运行过程的管理。

输入、输出与运行条件：本块输入是已读入的taskname；输出是执行任务后回到加载器。任务不返回时，调用者也停留在这次调用中。

1. P1：查找、检查、直接调用、返回提示

P1:init/main.c · L129–L135 · 完整的调用点逻辑块
P1原实现：对比起点

```
129         uint64_t entry=load_task_img(taskname);
130         if(entry==0){
131             bios_putstr("Task not found\n\r");
132             continue;
133         }
134         ((void (*)(void))entry)();
135         bios_putstr("\n\rTask finished, back to loader\n\r");
```

按原始行号逐句理解

L129

```
        uint64_t entry=load_task_img(taskname);
```

load_task_img按名字读取任务字节，并返回入口地址。entry是一个数值地址，还没有执行任务。

L130–133

```
        if(entry==0){
            bios_putstr("Task not found\n\r");
            continue;
        }
```

入口为0表示查找失败。打印错误后continue回到外层输入循环，避免跳到地址0。该分支不创建任务对象。

L134

```
        ((void (*)(void))entry)();
```

把入口转换成“无参数、无返回值”的函数指针并立即调用。当前CPU沿一条调用链进入任务，尚无PCB切换、独立调度或特权级切换。

L135

```
        bios_putstr("\n\rTask finished, back to loader\n\r");
```

只有上面的调用正常返回，这句才执行。它证明当前任务交回了控制权，不能让一个不返回的任务自动让出CPU。

把执行过程手算一遍

1. P1：输入data → load_task_img → 任务crt0 → 任务main → 返回crt0 → 返回内核调用点。
2. P2：加载A/B → 分别建PCB和初始栈 → 进入就绪队列 → 调度A → A主动让出或被时钟打断 → 调度B → 以后恢复A原来的位置。

应该看到什么，由哪里保证

- P1可看到Task finished；P2课程任务多为无限循环，没有“每次回到名字提示”的设计。
- P2所谓“多个任务运行”，在当前单核上是交替使用CPU，不是同一时刻执行两条任务指令。

对应本次验证：本章用于解释原代码；P2运行证据在最后的Task1—4验收表。（详细范围和结果见末尾）。

> **理解边界：**P1的sched.h已有部分结构声明，不能说P1完全没有PCB这几个字；缺的是实际接入执行流程的实现。

Task 1 · 数据模型

02 先记住任务，再记住两种现场

> **这一步要完成什么**
>
> 给每个任务一个稳定的PCB，并区分“异常完整现场”和“C调用切换现场”。

与P1／前一步的关系：P1中任务主要靠普通调用的ra和当前栈返回；P2必须在暂停之后记住下次从哪里继续。PCB原有部分字段来自框架，新增trap_t0和user_stack_base辅助异常入口与指针检查。

输入、输出与运行条件：输入是任务状态和寄存器的约定；输出是C与汇编都能按固定偏移访问的内存结构。

1. 两种现场、任务状态和PCB的完整定义

P2:include/os/sched.h · L38–L90 · 完整函数／连续实现块
相对P1修改的文件；课程框架基础上修改／补全

```
38 typedef struct regs_context
39 {
40     /* Saved main processor registers.*/
41     reg_t regs[32];
42 
43     /* Saved special registers. */
44     reg_t sstatus;
45     reg_t sepc;
46     reg_t sbadaddr;
47     reg_t scause;
48 } regs_context_t;
49 
50 /* used to save register infomation in switch_to */
51 typedef struct switchto_context
52 {
53     /* Callee saved registers.*/
54     reg_t regs[14];
55 } switchto_context_t;
56 
57 typedef enum {
58     TASK_BLOCKED,
59     TASK_RUNNING,
60     TASK_READY,
61     TASK_EXITED,
62 } task_status_t;
63 
64 /* Process Control Block */
65 typedef struct pcb
66 {
67     /* register context */
68     // NOTE: this order must be preserved, which is defined in regs.h!!
69     reg_t kernel_sp;
70     reg_t user_sp;
+ 71     reg_t trap_t0; /* temporary saved before trap stack selection */
+ 72     ptr_t user_stack_base;
```

```
    /* previous, next pointer */
    list_node_t list;

    /* process id */
    pid_t pid;

    /* BLOCK | READY | RUNNING */
    task_status_t status;

    /* cursor position */
    int cursor_x;
    int cursor_y;

    /* time(seconds) to wake up sleeping PCB */
+   uint64_t wakeup_time; /* absolute rdtime ticks */

} pcb_t;
```

按原始行号逐句理解

L38–39

```
typedef struct regs_context
{
```

定义异常现场结构。这里先写数据格式，尚未执行保存指令；真正写入这些槽位的是SAVE_CONTEXT。

L40–41

```
    /* Saved main processor registers.*/
    reg_t regs[32];
```

regs[32]为x0到x31各留一个reg_t槽。在RV64每槽8字节；数组下标2对应sp、4对应tp、10对应a0、17对应a7。

L43–48

额外保存sstatus、sepc、sbadaddr和scause。它们分别描述返回模式/中断状态、恢复PC、异常相关地址和原因；成员sbadaddr在本代码中存的是CSR_STVAL。总计36×8=288字节。

L50–52

```
/* used to save register infomation in switch_to */
typedef struct switchto_context
{
```

定义另一种用于switch_to的现场。它服务于C函数调用边界，不是上面异常现场的同义词。

L53–55

```
    /* Callee saved registers.*/
    reg_t regs[14];
} switchto_context_t;
```

14个槽位保存ra、sp、s0到s11，总计112字节。普通C调用允许临时/参数寄存器被改写，所以这个帧无需重复保存全部32个寄存器；从U态陷入时另有完整异常帧兜住用户寄存器。

L57–62

列出阻塞、运行、就绪和退出状态。枚举中有EXITED不表示本包已经实现退出系统调用；实际主线围绕前三种状态工作。

L64–66

```
/* Process Control Block */
typedef struct pcb
{
```

定义PCB：一份PCB对应一个任务。改变状态或移动链表节点时操作的是同一个对象，不是复制出多个任务。

L67–70

```
    /* register context */
    // NOTE: this order must be preserved, which is defined in regs.h!!
    reg_t kernel_sp;
    reg_t user_sp;
```

kernel_sp记录可恢复的内核栈位置，user_sp记录任务栈位置。汇编用固定偏移0和8访问，不能随意调换成员顺序。

L71

```
    reg_t trap_t0; /* temporary saved before trap stack selection */
```

trap_t0给异常入口暂存原t0。异常入口先要用t0计算切栈，却又不能丢掉用户原来的t0，所以需要这块临时空间。

L72

```
    ptr_t user_stack_base;
```

user_stack_base记录用户栈低地址，k_write用它检查字符串指针范围；它本身不会建立页表隔离。

L74–75

```
    /* previous, next pointer */
    list_node_t list;
```

list是内嵌双向链表节点。队列存节点地址，list_entry再反推出PCB地址。一份PCB只有这个节点，不能同时挂进就绪和等待两个队列。

L77–78

```
    /* process id */
    pid_t pid;
```

pid保存任务编号。当前实现pid0是idle，普通任务从1递增；数字不会自动赋予调度能力。

L80–81

```
    /* BLOCK | READY | RUNNING */
    task_status_t status;
```

status让调度器知道任务能否回到就绪队列。锁和sleep必须先标BLOCKED，才能避免被当成普通运行任务重新排队。

L83–85

```
    /* cursor position */
    int cursor_x;
    int cursor_y;
```

每任务独立记录屏幕光标位置，切换任务后仍能在各自输出行更新。它不是显卡硬件上下文。

L87–90

```
    /* time(seconds) to wake up sleeping PCB */
    uint64_t wakeup_time; /* absolute rdtime ticks */

} pcb_t;
```

wakeup_time记录到期时刻。注意实际单位是绝对rdtime ticks，虽然上一行旧注释写seconds；秒数是在do_sleep中乘time_base后转换的。

2. current_running与任务集合

P2:include/os/sched.h · L99–L104 · 完整声明块
相对P1修改的文件；课程框架基础上修改／补全

```
+   99 register pcb_t * current_running asm("tp");
   100 extern pid_t process_id;
   101
   102 extern pcb_t pcb[NUM_MAX_TASK];
   103 extern pcb_t pid0_pcb;
   104 extern const ptr_t pid0_stack;
```

按原始行号逐句理解

L99

```
register pcb_t * current_running asm("tp");
```

把current_running绑定到RISC-V的tp寄存器。内核可以直接通过tp取得当前PCB，不能把它误当成普通的内存全局变量；U态的tp在异常入口要另外保存。

L100

```
extern pid_t process_id;
```

extern声明pid分配计数器，真正定义在sched.c。每次process_id++给新任务取一个不同编号。

L102–104

```
extern pcb_t pcb[NUM_MAX_TASK];
extern pcb_t pid0_pcb;
extern const ptr_t pid0_stack;
```

声明普通任务数组、pid0 PCB与idle栈顶。extern不会在每个包含头文件的文件中重新分配这些对象。

3. 编译期检查：C结构体必须与汇编约定一致

P2:init/main.c · L24–L29 · 完整断言块
相对P1修改的文件；课程框架基础上修改／补全

```
+   24 _Static_assert(sizeof(regs_context_t) == OFFSET_SIZE, "trap ABI size");
+   25 _Static_assert(sizeof(switchto_context_t) == SWITCH_TO_SIZE, "switch ABI size");
+   26 _Static_assert(__builtin_offsetof(pcb_t, kernel_sp) == PCB_KERNEL_SP, "kernel_sp ABI");
+   27 _Static_assert(__builtin_offsetof(pcb_t, user_sp) == PCB_USER_SP, "user_sp ABI");
+   28 _Static_assert(__builtin_offsetof(pcb_t, trap_t0) == PCB_TRAP_T0, "trap scratch ABI");
```

```
+   29 _Static_assert(sizeof(task_info_t) == 48, "disk task metadata ABI");
```

按原始行号逐句理解

L24

```
_Static_assert(sizeof(regs_context_t) == OFFSET_SIZE, "trap ABI size");
```

确认异常帧大小等于OFFSET_SIZE，即288字节；布局一旦改错，在编译阶段失败，而不是运行后才错恢复寄存器。

L25

```
_Static_assert(sizeof(switchto_context_t) == SWITCH_TO_SIZE, "switch ABI size");
```

确认切换帧大小等于112字节。

L26–28

```
_Static_assert(__builtin_offsetof(pcb_t, kernel_sp) == PCB_KERNEL_SP, "kernel_sp ABI");
_Static_assert(__builtin_offsetof(pcb_t, user_sp) == PCB_USER_SP, "user_sp ABI");
_Static_assert(__builtin_offsetof(pcb_t, trap_t0) == PCB_TRAP_T0, "trap scratch ABI");
```

分别校验kernel_sp、user_sp、trap_t0的实际C成员偏移，与汇编PCB_KERNEL_SP/USER_SP/TRAP_T0一致。

L29

```
_Static_assert(sizeof(task_info_t) == 48, "disk task metadata ABI");
```

校验磁盘任务记录为48字节。它与PCB不同：task_info_t描述程序在镜像中的位置，PCB描述程序运行中的状态。

把执行过程手算一遍

1. 先问“这是什么对象”：task_info_t回答代码在哪，pcb_t回答谁正在跑，regs_context_t回答被打断时CPU是什么样。
2. 一次ecall可以在A的内核栈上先保存288字节用户异常帧，再在调度切换时另压112字节切换帧；这两个帧各有用途。

应该看到什么，由哪里保证

- 寄存器探针要求任务经过抢占和系统调用后仍保持正确用户寄存器；字段偏移与保存/恢复指令共同保证。

对应本次验证：stage4（详细范围和结果见末尾）。

> **理解边界：**仅定义结构体没有实现调度；后续初始化、切栈和状态/队列操作缺一不可。 当前系统没有分页隔离或SMP，不能把PCB称作已经具备完整Linux进程隔离。

Task 1 · 就绪队列

03 用双向链表组织“谁可以运行”

> **这一步要完成什么**
>
> 实现排队、取队首、脱链，并能从节点地址找回PCB。

与P1／前一步的关系：P1没有在实际执行流程中维护就绪队列。P2的调度、sleep和锁等待共用这组基础操作。

输入、输出与运行条件：head是哨兵节点，node是PCB内部的list成员；输出是指针关系变化，不分配或复制任务。

1. 链表节点、头部和所有关键操作

P2:include/os/list.h · L43–L82 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+   43 typedef struct list_node
+   44 {
+   45     struct list_node *next, *prev;
+   46 } list_node_t;
+   47
+   48 typedef list_node_t list_head;
+   49
+   50 // LIST_HEAD is used to define the head of a list.
+   51 #define LIST_HEAD(name) struct list_node name = {&(name), &(name)}
+   52
+   53 static inline void list_init(list_head *head)
+   54 {
+   55     head->next = head->prev = head;
+   56 }
+   57 static inline int list_empty(const list_head *head)
+   58 {
+   59     return head->next == head;
+   60 }
+   61 static inline void list_add_tail(list_node_t *node, list_head *head)
+   62 {
+   63     node->prev = head->prev;
+   64     node->next = head;
+   65     head->prev->next = node;
+   66     head->prev = node;
+   67 }
+   68 static inline void list_del(list_node_t *node)
+   69 {
+   70     node->prev->next = node->next;
+   71     node->next->prev = node->prev;
+   72     list_init(node);
+   73 }
+   74 static inline list_node_t *list_pop_front(list_head *head)
+   75 {
+   76     if (list_empty(head)) return NULL;
+   77     list_node_t *node = head->next;
+   78     list_del(node);
```

```
+   79     return node;
+   80 }
+   81 #define list_entry(ptr, type, member) \
+   82     ((type *)((char *)(ptr) - __builtin_offsetof(type, member)))
```

按原始行号逐句理解

L43–46

```
typedef struct list_node
{
    struct list_node *next, *prev;
} list_node_t;
```

每个节点保存next和prev，分别指向后继和前驱；typedef取别名list_node_t。

L48

```
typedef list_node_t list_head;
```

队列头与普通节点采用同一种结构，但队列头只作哨兵，没有对应的普通任务。

L50–51

```
// LIST_HEAD is used to define the head of a list.
#define LIST_HEAD(name) struct list_node name = {&(name), &(name)}
```

LIST_HEAD定义并初始化一个队列头，让next和prev都指向自身。调用这个宏是在创建队列头，不是在创建PCB。

L53–56

```
static inline void list_init(list_head *head)
{
    head->next = head->prev = head;
}
```

list_init同样把已有节点设为自环。它既可初始化空队列头，也用于标记一个节点当前不在队列中。

L57–60

```
static inline int list_empty(const list_head *head)
{
    return head->next == head;
}
```

空队列的第一个节点就是头自己，所以比较head->next与head即可判空。

L61–62

```
static inline void list_add_tail(list_node_t *node, list_head *head)
{
```

定义尾插操作；调用者应确保node当前已脱链，不能把同一节点重复加入队列。

L63

```
    node->prev = head->prev;
```

新节点的前驱设为原队尾。

L64

```
    node->next = head;
```

新节点的后继设为哨兵头，准备插在原队尾和头之间。

L65

```
    head->prev->next = node;
```

把原队尾的next连向新节点。这步使用的head->prev仍必须是原队尾。

L66–67

```
    head->prev = node;
}
```

最后更新头的prev为新队尾。四条连接关系至此配对，不能先改head->prev再执行第65行。

L68–69

```
static inline void list_del(list_node_t *node)
{
```

定义从当前位置删除节点的操作；传入的是节点，不是队列头或PCB值的副本。

L70

```
    node->prev->next = node->next;
```

让前驱跳过本节点，指向原后继。

L71

```
    node->next->prev = node->prev;
```

让后继的prev回接原前驱，完成双向脱链。

L72–73

```
    list_init(node);
}
```

删除后把本节点重新设成自环，后面的list_empty(node)就能判定它已脱队。

L74–75

```
static inline list_node_t *list_pop_front(list_head *head)
{
```

定义弹出队首。弹出的节点将交给调度器，不会自动执行任务代码。

L76

```
    if (list_empty(head)) return NULL;
```

判空后返回NULL，防止把哨兵当普通任务。

L77

```
    list_node_t *node = head->next;
```

暂存第一个真实节点地址。

L78

```
    list_del(node);
```

调用list_del脱链，同时恢复该节点的自环状态。

L79–80

```
    return node;
}
```

把已脱链节点返回给调用者；调度器之后用它取得PCB。

L81–82

```
#define list_entry(ptr, type, member) \
    ((type *)((char *)(ptr) - __builtin_offsetof(type, member)))
```

list_entry用成员地址减去成员在结构体内的偏移，回到PCB起始地址。先转char*使减法按字节进行；它不是查表或分配内存。

把执行过程手算一遍

1. 从空head开始：加入A得到head↔A↔head；再加入B得到head↔A↔B↔head。
2. pop_front取出A后，队列为head↔B↔head，A自身为自环。以后A被调度出去时才可能再入队。

应该看到什么，由哪里保证

- Task1的A、B以FIFO方式轮转，前提是尾插与头取保持正确；重复入队、单向断链都会破坏这个现象。

对应本次验证：stage1（详细范围和结果见末尾）。

> **理解边界：**链表里挂的是PCB内部的节点；同一PCB不能同时在ready_queue和某个锁等待队列中。

Task 1 · 创建任务

04 加载程序只是第一步，还要分栈并建PCB

> **这一步要完成什么**
>
> 给多个任务分配独立的栈，构造运行记录，然后统一放入就绪队列。

与P1／前一步的关系：P1加载后直接调用入口。P2仍按名字装入程序，但先给每个程序准备独立执行条件。

输入、输出与运行条件：init_pcb读取已填好的tasks数组，在S态内核启动阶段执行；此时中断尚未开放。结果是pid0成为当前任务，其余任务处于READY。

1. 页分配器：为栈取得不同的内存区间

P2:kernel/mm/mm.c · L3–L20 · 完整函数／连续实现块
相对P1新增的文件；课程框架保留的关键依赖

```
+   3 static ptr_t kernMemCurr = FREEMEM_KERNEL;
+   4 static ptr_t userMemCurr = FREEMEM_USER;
+   5
+   6 ptr_t allocKernelPage(int numPage)
+   7 {
+   8     // align PAGE_SIZE
+   9     ptr_t ret = ROUND(kernMemCurr, PAGE_SIZE);
+  10     kernMemCurr = ret + numPage * PAGE_SIZE;
+  11     return ret;
+  12 }
+  13
+  14 ptr_t allocUserPage(int numPage)
+  15 {
+  16     // align PAGE_SIZE
+  17     ptr_t ret = ROUND(userMemCurr, PAGE_SIZE);
+  18     userMemCurr = ret + numPage * PAGE_SIZE;
+  19     return ret;
+  20 }
```

按原始行号逐句理解

L3–4

```
static ptr_t kernMemCurr = FREEMEM_KERNEL;
static ptr_t userMemCurr = FREEMEM_USER;
```

分别记录内核和用户可分配区域的当前位置。static使这两个变量在文件内部可见，值会在多次函数调用之间保留。

L6–7

```
ptr_t allocKernelPage(int numPage)
{
```

allocKernelPage接收页数，返回该次分配区域的低地址；它不是返回栈顶。

L8–9

```
    // align PAGE_SIZE
    ptr_t ret = ROUND(kernMemCurr, PAGE_SIZE);
```

ROUND把当前位置向上对齐到PAGE_SIZE=4096字节边界。

L10

```
    kernMemCurr = ret + numPage * PAGE_SIZE;
```

推进内核分配游标，下一次分配从更高地址继续。当前是简单递增分配，没有回收或完整越界管理。

L11–12

```
    return ret;
}
```

返回区域起始地址。调用者还需加PAGE_SIZE，才得到一页栈的初始高地址栈顶。

L14–15

```
ptr_t allocUserPage(int numPage)
{
```

用户页分配器接口与内核页类似，但使用另一条分配游标。

L16–17

```
    // align PAGE_SIZE
    ptr_t ret = ROUND(userMemCurr, PAGE_SIZE);
```

对齐用户区域起点；两类游标分别从FREEMEM_KERNEL与FREEMEM_USER开始。

L18

```
    userMemCurr = ret + numPage * PAGE_SIZE;
```

按申请页数推进用户区域的游标。

L19–20

```
    return ret;
}
```

返回用户区低地址；“用户页”在这里是用途命名，没有建立页表映射或隔离。

2. init_pcb：完整创建流程

P2:init/main.c · L146–L171 · 完整函数／连续实现块
相对P1修改的文件；课程框架基础上修改／补全

```
+ 146 static void init_pcb(void)
+ 147 {
+ 148     list_init(&ready_queue);
+ 149     list_init(&sleep_queue);
+ 150     list_init(&pid0_pcb.list);
+ 151     pid0_pcb.status = TASK_RUNNING;
+ 152     current_running = &pid0_pcb;
+ 153     __asm__ volatile ("csrw sscratch, %0" :: "r"(current_running) : "memory");
+ 154     for (int i = 0; i < task_num; ++i) {
+ 155         pcb_t *task = &pcb[i];
```

```
+ 156         bzero(task, sizeof(*task));
+ 157         task->pid = process_id++;
+ 158         task->status = TASK_READY;
+ 159         task->cursor_y = i * 2;
+ 160         list_init(&task->list);
+ 161         ptr_t ktop = allocKernelPage(1) + PAGE_SIZE;
+ 162         task->user_stack_base = allocUserPage(1);
+ 163         ptr_t utop = task->user_stack_base + PAGE_SIZE;
+ 164         uint64_t entry = load_task_img(tasks[i].task_name);
+ 165         assert(entry);
+ 166         init_pcb_stack(ktop, utop, entry, task);
+ 167         list_add_tail(&task->list, &ready_queue);
+ 168         printk("[LOAD] pid=%d task=%s entry=0x%lx\n\r", task->pid,
+ 169                tasks[i].task_name, entry);
+ 170     }
+ 171 }
```

按原始行号逐句理解

L146–147

```
static void init_pcb(void)
{
```

定义启动阶段的PCB初始化函数，不是运行时exec系统调用。

L148–149

```
    list_init(&ready_queue);
    list_init(&sleep_queue);
```

把就绪与睡眠队列初始化为空。随后只有已准备好的任务才可入队。

L150

```
    list_init(&pid0_pcb.list);
```

初始化idle任务的节点为自环。

L151

```
    pid0_pcb.status = TASK_RUNNING;
```

标记pid0正在运行：当前执行的正是内核main这条启动调用链。

L152

```
    current_running = &pid0_pcb;
```

把tp/current_running设为pid0 PCB。第一次switch_to才能把main现场保存到正确对象。

L153

```
    __asm__ volatile ("csrw sscratch, %0" :: "r"(current_running) : "memory");
```

把sscratch也设为当前PCB，供异常入口可靠地找到内核数据。memory约束阻止编译器把相关内存操作随意跨越这条汇编。

L154

```
    for (int i = 0; i < task_num; ++i) {
```

按任务表的顺序为每个镜像任务创建PCB；不是等待键盘输入后逐个执行。

L155

```
        pcb_t *task = &pcb[i];
```

task指向pcb数组中的当前元素，后面的->操作都在填写这一个PCB。

L156

```
        bzero(task, sizeof(*task));
```

先清零，避免游标或字段残留。清零本身不能替代下面对状态和链表的正确初始化。

L157

```
        task->pid = process_id++;
```

分配pid；process_id初始为1，pid0保留给idle。

L158

```
        task->status = TASK_READY;
```

新任务处于READY，表示可调度但尚未占用CPU。

L159

```
        task->cursor_y = i * 2;
```

给不同任务安排不同屏幕行，方便观察交替运行。

L160

```
        list_init(&task->list);
```

初始化任务节点自环，确保加入队列前不隶属任何旧队列。

L161

```
        ptr_t ktop = allocKernelPage(1) + PAGE_SIZE;
```

为内核调用链分配一页，并将低地址加4096得到内核栈顶ktop。

L162

```
        task->user_stack_base = allocUserPage(1);
```

为任务另分一页并保存低地址，后续字符串范围检查也用到它。

L163

```
        ptr_t utop = task->user_stack_base + PAGE_SIZE;
```

由用户栈低地址得到栈顶utop；RISC-V栈向低地址增长。

L164

```
        uint64_t entry = load_task_img(tasks[i].task_name);
```

沿用按名加载器，把任务代码读到它链接时约定的入口区域。还没有开始执行任务。

L165

```
        assert(entry);
```

如果加载失败立即断言，不能把0当成启动PC。

L166

```
        init_pcb_stack(ktop, utop, entry, task);
```

下一章的函数在内核栈顶下摆出两个初始帧，决定该任务第一次如何开始。

L167

```
        list_add_tail(&task->list, &ready_queue);
```

准备齐全后才尾插到ready_queue；入队不等于当前立刻切过去。

L168–169

```
        printk("[LOAD] pid=%d task=%s entry=0x%lx\n\r", task->pid,
               tasks[i].task_name, entry);
```

打印pid、名字和入口，是加载初始化的证据；只看到LOAD不证明任务已实际运行。

L170–171

```
    }
}
```

循环完成后所有任务已就绪，随后main继续初始化时钟、锁、异常等设施。

把执行过程手算一遍

1. 假设只选A/B：pid0在运行，A获得一组ktop/utop与pid1，B获得另一组栈与pid2，ready_queue最终为[A,B]。
2. TASK_SIZE=64KiB是映像槽位限制，PAGE_SIZE=4KiB是这里每次分栈的大小，两者不是同一个单位。

应该看到什么，由哪里保证

- 启动会出现每个任务的[LOAD]记录；真正的计数更新还要经过调度与首次sret路径。

对应本次验证：stage1 / stage4（详细范围和结果见末尾）。

> **理解边界：**把allocKernelPage的返回值直接当栈顶会把首次压栈写到区域之前。 每个任务栈分开不意味着有完整虚拟地址隔离。

Task 1 / 3 · 第一次启动

05 伪造一份“以后恢复时会用到”的现场

这一步要完成什么

让第一次启动也走同一条恢复路径，而不是为新任务另写一个普通入口调用。

与P1／前一步的关系：P1的crt0保存调用者ra并返回加载器。P2没有这样的普通调用者，需要自己提供首次ret和sret的去向。

输入、输出与运行条件：输入ktop/utop/entry和PCB；输出PCB.kernel_sp指向一份可由switch_to恢复的初始栈。此函数在S态启动阶段运行。

1. init_pcb_stack：完整构造两个初始帧

P2:init/main.c · L124–L144 · 完整函数／连续实现块
相对P1修改的文件；课程框架基础上修改／补全

```
+ 124 static void init_pcb_stack(ptr_t ktop, ptr_t utop, ptr_t entry, pcb_t *task)
+ 125 {
+ 126     regs_context_t *regs = (regs_context_t *)(ktop - sizeof(*regs));
+ 127     switchto_context_t *sw = (switchto_context_t *)((ptr_t)regs - sizeof(*sw));
+ 128     bzero(regs, sizeof(*regs));
+ 129     bzero(sw, sizeof(*sw));
+ 130     regs->regs[2] = utop;
+ 131     regs->sepc = entry;
+ 132 #if LAB_STAGE >= 3
+ 133     /* SPP=0 returns to U mode; SPIE controls SIE after sret. */
+ 134     regs->sstatus = SR_SPIE;
+ 135 #else
+ 136     /* Cooperative tasks share S mode and call the jump table directly. */
+ 137     regs->regs[4] = (ptr_t)task;
+ 138     regs->sstatus = SR_SPP;
+ 139 #endif
+ 140     sw->regs[0] = (ptr_t)ret_from_exception;
+ 141     sw->regs[1] = (ptr_t)sw;
+ 142     task->kernel_sp = (ptr_t)sw;
+ 143     task->user_sp = utop;
+ 144 }
```

按原始行号逐句理解

L124–125

```
static void init_pcb_stack(ptr_t ktop, ptr_t utop, ptr_t entry, pcb_t *task)
{
```

参数分别是内核栈顶、任务栈顶、任务crt0入口和PCB指针，都是给未来执行准备数据。

L126

```
    regs_context_t *regs = (regs_context_t *)(ktop - sizeof(*regs));
```

在ktop以下预留288字节异常帧，regs指向它的低地址。

L127

```
    switchto_context_t *sw = (switchto_context_t *)((ptr_t)regs - sizeof(*sw));
```

在异常帧下面再预留112字节切换帧，sw指向它的低地址；两块加起来400字节。

L128–129

```
    bzero(regs, sizeof(*regs));
    bzero(sw, sizeof(*sw));
```

清零两帧，让没有单独设定的寄存器槽也有明确初值。

L130

```
    regs->regs[2] = utop;
```

regs[2]对应x2/sp，恢复异常现场后任务会使用utop，而不是内核正在构造现场的sp。

L131

```
    regs->sepc = entry;
```

sepc是sret的跳转目标，设为入口entry。它通常指任务crt0的_start，而不是直接跳到用户C main。

L132–134

```
#if LAB_STAGE >= 3
    /* SPP=0 returns to U mode; SPIE controls SIE after sret. */
    regs->sstatus = SR_SPIE;
```

阶段3/4把SPP保持为0，使sret进入U态；SPIE置1，使sret之后的SIE按约定恢复。此处设的是帧中将来的状态，不是现在立即开中断。

L135–138

```
#else
    /* Cooperative tasks share S mode and call the jump table directly. */
    regs->regs[4] = (ptr_t)task;
    regs->sstatus = SR_SPP;
```

阶段1/2任务留在S态。保存的tp设为自己的PCB，SPP置1，所以任务能通过内核跳转表直接调用服务。

L139

```
#endif
```

编译条件结束；一次构建只保留上面某一分支。

L140

```
    sw->regs[0] = (ptr_t)ret_from_exception;
```

切换帧的ra设为ret_from_exception。首次switch_to最后执行ret时，就会到这个统一恢复入口。

L141

```
    sw->regs[1] = (ptr_t)sw;
```

切换帧中保存的sp设为sw本身，配合switch_to尾部恢复sp再加112，准确落到异常帧起点。

L142

```
    task->kernel_sp = (ptr_t)sw;
```

PCB记住切换帧的地址，调度器将来从这里恢复该任务。

L143–144

```
    task->user_sp = utop;
}
```

记住任务栈顶，初始化结束。这里只写内存，没有执行entry处的代码。

2. P2任务crt0：完整入口与main返回后的兜底

P2:arch/riscv/crt0/crt0.S · L1–L18 · 完整函数／连续实现块
相对P1修改的文件；课程框架基础上修改／补全

```
    1 #include <asm.h>
    2 .section ".entry_function","ax"
    3 ENTRY(_start)
+   4     /* The kernel already supplied a private aligned stack. */
    5     la t0, __bss_start
    6     la t1, __BSS_END__
+   7 1:  bgeu t0, t1, 2f
+   8     sb zero, 0(t0)
+   9     addi t0, t0, 1
+  10     j 1b
+  11 2:  call main
+  12     /* P2 tasks normally never return; avoid returning to a fake caller. */
+  13 3:  call sys_yield
+  14     j 3b
   15     /*************************************************************/
+  16     /* Do not touch this comment. Reserved for future projects. */
+  17     /*************************************************************/
+  18 ENDPROC(_start)
```

按原始行号逐句理解

L1

```
#include <asm.h>
```

包含ENTRY等汇编宏的定义。

L2

```
.section ".entry_function","ax"
```

把入口放在可执行的.entry_function节，链接脚本会把它排到任务映像前面。

L3–4

```
ENTRY(_start)
    /* The kernel already supplied a private aligned stack. */
```

导出_start。内核已经通过初始帧提供独立任务栈，因此不再保存一个不存在的“加载器调用者”。

L5–6

```
    la t0, __bss_start
    la t1, __BSS_END__
```

取得BSS起止符号，准备把任务的未初始化数据清零。

L7

```
1:  bgeu t0, t1, 2f
```

无符号比较：当指针到达或超过终点，就结束清零。

L8

```
    sb zero, 0(t0)
```

每轮清一个字节，既不要求BSS长度是4或8的倍数，也不会多写尾部。

L9–10

```
    addi t0, t0, 1
    j 1b
```

地址加1，再跳回最近的数字标签1。汇编1b表示向后找1号标签。

L11

```
2:  call main
```

清零完成后普通call进入任务C main。这一次main确实有crt0作为调用者。

L12–14

```
    /* P2 tasks normally never return; avoid returning to a fake caller. */
3:  call sys_yield
    j 3b
```

main即使意外返回，也循环调用sys_yield并让出CPU；没有P1那条“恢复ra后返回名字加载器”的路径，也不等于已经实现任务退出。

L15–17

```
    /*************************************************************/
    /* Do not touch this comment. Reserved for future projects. */
    /*************************************************************/
```

保留给后续实验的注释，不生成运行逻辑。

L18

```
ENDPROC(_start)
```

结束入口符号定义。

把执行过程手算一遍

1. 若ktop=0x50503000，则regs=0x50502ee0、sw=0x50502e70。先恢复sw并加112，sp正好变为regs；再从regs恢复任务sp=utop。
2. 完整链：调度器选中任务 → switch_to取出预制ra → ret_from_exception → RESTORE_CONTEXT → sret按sepc进入crt0 → call main。

应该看到什么，由哪里保证

- 首次调度能直接进入两个新任务，之后又能恢复旧任务；两种情况共用switch_to尾部的恢复代码。

对应本次验证：stage1 / stage3 / stage4（详细范围和结果见末尾）。

> **理解边界：**ra、sepc、sp是三种不同信息：ret读ra，sret读sepc，sp决定从哪块栈取数据。 把sw.ra直接设为用户main会绕过现场恢复与模式切换。

Task 1 · 选择下一个任务

06 FIFO轮转：该回队尾的才回队尾

这一步要完成什么

用状态和队列共同决定下一个任务，并在无人就绪时返回idle。

与P1／前一步的关系：P1没有在任务之间选择的调度器。P2把yield、阻塞和定时中断最终汇聚到do_scheduler。

输入、输出与运行条件：进入时current_running指向当前PCB，队列保持双向链表不变量。函数可能切到别的任务，直到将来恢复本任务才继续执行末尾。

1. 调度器全局对象与完整do_scheduler

P2:kernel/sched/sched.c · L8–L37 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+   8 pcb_t pcb[NUM_MAX_TASK];
+   9 const ptr_t pid0_stack = INIT_KERNEL_STACK + PAGE_SIZE;
+  10 pcb_t pid0_pcb = { .pid = 0, .kernel_sp = INIT_KERNEL_STACK + PAGE_SIZE,
+  11                    .user_sp = INIT_KERNEL_STACK + PAGE_SIZE };
+  12 LIST_HEAD(ready_queue);
+  13 LIST_HEAD(sleep_queue);
+  14 pid_t process_id = 1;
+  15
+  16 void do_scheduler(void)
+  17 {
+  18     uint64_t flags = irq_save();
+  19     check_sleeping();
+  20     /**********************************************************/
+  21     /* Do not touch this comment. Reserved for future projects. */
+  22     /**********************************************************/
+  23     pcb_t *prev = current_running;
+  24     if (prev->status == TASK_RUNNING && prev != &pid0_pcb) {
+  25         prev->status = TASK_READY;
+  26         list_add_tail(&prev->list, &ready_queue);
+  27     }
+  28     list_node_t *node = list_pop_front(&ready_queue);
+  29     pcb_t *next = node ? list_entry(node, pcb_t, list) : &pid0_pcb;
+  30     if (prev == &pid0_pcb && next != prev) prev->status = TASK_READY;
+  31     next->status = TASK_RUNNING;
+  32     if (prev != next) {
+  33         /* switch_to installs tp and sscratch atomically with IRQs disabled. */
+  34         switch_to(prev, next);
+  35     }
+  36     irq_restore(flags);
+  37 }
```

按原始行号逐句理解

L8

```
pcb_t pcb[NUM_MAX_TASK];
```

为普通任务保留PCB数组，容量由NUM_MAX_TASK控制。

L9

```
const ptr_t pid0_stack = INIT_KERNEL_STACK + PAGE_SIZE;
```

idle栈顶与head.S设定的启动栈顶一致，都是INIT_KERNEL_STACK+PAGE_SIZE。

L10–11

```
pcb_t pid0_pcb = { .pid = 0, .kernel_sp = INIT_KERNEL_STACK + PAGE_SIZE,
                   .user_sp = INIT_KERNEL_STACK + PAGE_SIZE };
```

初始化pid0的栈字段。第一次切换保存的就是当前main执行上下文。

L12–13

```
LIST_HEAD(ready_queue);
LIST_HEAD(sleep_queue);
```

分别定义自环的就绪队列头与睡眠队列头。

L14

```
pid_t process_id = 1;
```

普通pid从1开始递增。

L16–17

```
void do_scheduler(void)
{
```

调度入口；它不会创建新任务，而是在已存在的PCB之间选一个。

L18

```
    uint64_t flags = irq_save();
```

保存旧中断状态并关闭本地SIE，避免队列改到一半被时钟打断。

L19

```
    check_sleeping();
```

先唤醒已经到期的睡眠任务，使它们有机会参加这次选择。

L20–22

```
    /*************************************************************/
    /* Do not touch this comment. Reserved for future projects. */
    /*************************************************************/
```

保留注释，不是调度条件。

L23

```
    pcb_t *prev = current_running;
```

保存旧任务指针prev；switch_to会改变current_running/tp，因此不能以后再取来当旧任务。

L24

```
    if (prev->status == TASK_RUNNING && prev != &pid0_pcb) {
```

只有仍处于RUNNING的普通任务才回队尾。已被do_block改为BLOCKED的任务不满足条件；pid0也不参加普通轮转。

L25–26

```
        prev->status = TASK_READY;
        list_add_tail(&prev->list, &ready_queue);
```

把该普通任务改READY并尾插，实现轮转中的“用过一次，排到最后”。

L27

```
    }
```

条件分支结束。

L28

```
    list_node_t *node = list_pop_front(&ready_queue);
```

从就绪队首弹出一个节点；链表函数已把它脱队。

L29

```
    pcb_t *next = node ? list_entry(node, pcb_t, list) : &pid0_pcb;
```

三目表达式选择：有节点就取得其PCB，没节点则使用pid0，避免对NULL做list_entry。

L30

```
    if (prev == &pid0_pcb && next != prev) prev->status = TASK_READY;
```

当idle让位给普通任务时，把idle状态从RUNNING改为READY；它仍不在普通就绪队列里。

L31

```
    next->status = TASK_RUNNING;
```

选中的任务设为RUNNING，普通任务的节点已脱链，不再同时算就绪队列成员。

L32

```
    if (prev != next) {
```

如果选中的恰好还是当前任务，无需真的换栈。

L33–34

```
        /* switch_to installs tp and sscratch atomically with IRQs disabled. */
        switch_to(prev, next);
```

否则调用switch_to切换寄存器与栈；这个调用将来恢复当前任务时才从这里返回。

L35–37

```
    }
    irq_restore(flags);
}
```

返回当前任务后按它原先保存的flags恢复中断状态。不是无条件打开中断，因而可安全嵌套于已关中断的系统调用处理。

把执行过程手算一遍

1. A在RUNNING、队列[B]：A主动yield后先把A排到尾部，得到[B,A]；弹出B，队列剩[A]，切换到B。
2. B因锁阻塞时先标BLOCKED：调度器不会把B塞回ready_queue，只能让等待条件满足后的do_unblock把它放回。

应该看到什么，由哪里保证

- Task1两计数轮流前进；Task2等待锁的任务不继续执行临界区；无人可跑时选择pid0。三种现象都依赖本函数的状态判断。

对应本次验证：stage1 / stage2 / idle（详细范围和结果见末尾）。

> **理解边界：**无条件回队尾会把阻塞任务重新运行，是锁看似存在却无效的常见原因。 切换一次后原函数还可能在以后返回，不应按“调用后马上在同一任务继续”理解。

Task 1 · 真正切换CPU

07 switch_to：为什么一个ret能回到另一个任务

> **这一步要完成什么**
>
> 保存A的C调用现场，换成B的栈与身份，再恢复B先前保存的寄存器。

与P1／前一步的关系：P1的返回沿当前栈取回ra；P2主动更换sp，所以同一套恢复指令会从另一任务栈取出另一份ra。

输入、输出与运行条件：按RISC-V ABI，a0=prev、a1=next；调用时中断已关闭。成功后CPU继续执行next上次暂停的位置，或新任务的ret_from_exception。

1. switch_to和首次返回入口：完整实现

P2:arch/riscv/kernel/entry.S · L130–L170 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+ 130 ENTRY(switch_to)
+ 131     addi sp, sp, -SWITCH_TO_SIZE
+ 132     sd ra, 0(sp)
+ 133     sd sp, 8(sp)
+ 134     sd s0, 16(sp)
+ 135     sd s1, 24(sp)
+ 136     sd s2, 32(sp)
+ 137     sd s3, 40(sp)
+ 138     sd s4, 48(sp)
+ 139     sd s5, 56(sp)
+ 140     sd s6, 64(sp)
+ 141     sd s7, 72(sp)
+ 142     sd s8, 80(sp)
+ 143     sd s9, 88(sp)
+ 144     sd s10, 96(sp)
+ 145     sd s11, 104(sp)
+ 146     sd sp, PCB_KERNEL_SP(a0)
+ 147     mv tp, a1
+ 148     csrw CSR_SSCRATCH, tp
+ 149     ld sp, PCB_KERNEL_SP(tp)
+ 150     ld ra, 0(sp)
+ 151     ld s0, 16(sp)
+ 152     ld s1, 24(sp)
+ 153     ld s2, 32(sp)
+ 154     ld s3, 40(sp)
+ 155     ld s4, 48(sp)
+ 156     ld s5, 56(sp)
+ 157     ld s6, 64(sp)
+ 158     ld s7, 72(sp)
+ 159     ld s8, 80(sp)
+ 160     ld s9, 88(sp)
+ 161     ld s10, 96(sp)
+ 162     ld s11, 104(sp)
+ 163     ld sp, SWITCH_TO_SP(sp)
```

```
+ 164     addi sp, sp, SWITCH_TO_SIZE
+ 165     ret
+ 166 ENDPROC(switch_to)
+ 167 ENTRY(ret_from_exception)
+ 168     RESTORE_CONTEXT
+ 169     sret
+ 170 ENDPROC(ret_from_exception)
```

按原始行号逐句理解

L130

```
ENTRY(switch_to)
```

定义switch_to符号，供C调度器调用。

L131

```
    addi sp, sp, -SWITCH_TO_SIZE
```

在当前任务栈上向下预留112字节，sp现在指向切换帧低地址。

L132

```
    sd ra, 0(sp)
```

保存当前ra：它指向调度器中switch_to之后的位置。

L133

```
    sd sp, 8(sp)
```

保存此刻的sp，也就是这个切换帧自身的地址。最后恢复时仍需再加112回到压帧前位置。

L134–145

依次保存s0到s11，每个8字节，偏移16到104。C ABI要求被调用者保持这些寄存器；临时和参数寄存器由正常调用规则或外层异常帧负责。

L146

```
    sd sp, PCB_KERNEL_SP(a0)
```

把当前切换帧地址写到prev->kernel_sp。以后选回A时，就从这里找回它的调用链。

L147

```
    mv tp, a1
```

tp改为next PCB。从这一句开始，内核身份已切到B，不能再用tp指代A。

L148

```
    csrw CSR_SSCRATCH, tp
```

同步把sscratch改为B，确保下一次异常能可靠定位B的PCB。

L149

```
    ld sp, PCB_KERNEL_SP(tp)
```

从B的PCB取出它保存的内核sp。真正跨任务的分界就在这里：后续ld从B的栈读取。

L150

```
    ld ra, 0(sp)
```

恢复B的ra。若B已运行过，它是B上次调用switch_to后的地址；若是新任务，它是预制的ret_from_exception。

L151–162 从B的帧恢复s0到s11，与保存顺序和字节偏移逐项对应。漏一个会造成任务局部状态被另一个任务污染。

L163

```
    ld sp, SWITCH_TO_SP(sp)
```

先取出帧中保存的sp。RISC-V先用旧sp计算地址，再把读取值写入sp，所以该写法可行。

L164

```
    addi sp, sp, SWITCH_TO_SIZE
```

把sp加112，撤销切换帧。新任务此时sp正好来到预制异常帧起点；旧任务则回到上次压帧之前。

L165–166

```
    ret
ENDPROC(switch_to)
```

ret跳到恢复的ra；它并不知道“任务”这个概念，只按寄存器跳转。结束switch_to符号定义。

L167

```
ENTRY(ret_from_exception)
```

统一的异常返回符号，也是新任务首次ret的去向。

L168

```
    RESTORE_CONTEXT
```

展开RESTORE_CONTEXT，从异常帧恢复任务寄存器和sepc/sstatus；完整宏在后面的返回章节展示。

L169

```
    sret
```

sret按sepc决定下一条指令，并依据SPP/SPIE恢复模式和中断状态。它不是普通ret。

L170

```
ENDPROC(ret_from_exception)
```

结束返回入口符号。

把执行过程手算一遍

1. 把A的栈想成一本夹着A书签的书，B的栈是另一本书。切换不是复制所有内存，而是把A书签存回PCB，再换到B那本书继续读。
2. 第二次调度回A时，A的switch_to才返回到A的do_scheduler；B稍后也会经历同样过程。

应该看到什么，由哪里保证

- A/B反复yield后计数继续增长；更强的寄存器探针检查被保存寄存器没有串值。

对应本次验证：stage1 / stage4（详细范围和结果见末尾）。

> **理解边界：**只修改current_running而不换sp，不算上下文切换。 异常帧必须保存被打断时的全部用户寄存器；不能把这里的14槽规则直接套到抢占入口。

Task 1 · 主动让出CPU

08 接好跳转表，让yield到达调度器

> **这一步要完成什么**
>
> 让阶段1/2的S态任务可以调用调度服务，并用两个主动yield程序验证最小闭环。

与P1／前一步的关系：P1已有BIOS相关跳转表，主要用于串口和读盘。P2扩大服务表，并用返回long的k_*包装层统一参数约定；这不是把BIOS地址改成调度器。

输入、输出与运行条件：阶段1/2任务和内核都运行在S态。sys_yield经函数表直接调用k_yield，再进入do_scheduler；阶段3/4同一接口改走ecall。

1. yield服务包装：完整函数

P2:init/main.c · L32–L33 · 完整函数／连续实现块
相对P1修改的文件；课程框架基础上修改／补全

```
+   32 static long k_yield(long a, long b, long c, long d, long e)
+   33 { do_scheduler(); return 0; }
```

按原始行号逐句理解

L32

```
static long k_yield(long a, long b, long c, long d, long e)
```

定义统一的五个long参数、一个long返回值接口；本服务不使用这些参数，但表中其他服务可共用同一调用形式。

L33

```
{ do_scheduler(); return 0; }
```

执行调度。当前任务以后被恢复时，do_scheduler才返回，然后k_yield返回0给调用者。

2. init_jmptab：完整服务表初始化

P2:init/main.c · L92–L110 · 完整函数／连续实现块
相对P1修改的文件；课程框架基础上修改／补全

```
    92 static void init_jmptab(void)
    93 {
+   94     long (**table)() = (long (**)())KERNEL_JMPTAB_BASE;
+   95     table[CONSOLE_PUTSTR] = k_putstr;
+   96     table[CONSOLE_PUTCHAR] = k_putchar;
+   97     table[CONSOLE_GETCHAR] = k_getchar;
+   98     table[SD_READ] = k_sdread;
+   99     table[SD_WRITE] = k_sdwrite;
+  100     table[QEMU_LOGGING] = k_logging;
```

```
+ 101     table[SET_TIMER] = k_settimer;
+ 102     table[READ_FDT] = k_fdt;
+ 103     table[MOVE_CURSOR] = k_cursor;
+ 104     table[PRINT] = k_write;
+ 105     table[YIELD] = k_yield;
+ 106     table[MUTEX_INIT] = k_lock_init;
+ 107     table[MUTEX_ACQ] = k_lock_acq;
+ 108     table[MUTEX_RELEASE] = k_lock_rel;
+ 109     table[REFLUSH] = k_refresh;
+ 110 }
```

按原始行号逐句理解

L92–93

```
static void init_jmptab(void)
{
```

定义启动时填写跳转表的函数。

L94

```
    long (**table)() = (long (**)())KERNEL_JMPTAB_BASE;
```

把固定地址解释为函数指针表首地址。table本身是指向函数指针对象的指针；本版没有P1那处放在函数返回类型前、实际不起表项保护作用的volatile。

L95–97

```
    table[CONSOLE_PUTSTR] = k_putstr;
    table[CONSOLE_PUTCHAR] = k_putchar;
    table[CONSOLE_GETCHAR] = k_getchar;
```

登记字符串输出、字符输出、字符读取三个包装入口；赋值只是存地址，此时不会打印或读键盘。

L98–99

```
    table[SD_READ] = k_sdread;
    table[SD_WRITE] = k_sdwrite;
```

登记读盘和写盘包装入口。本版把SD_WRITE也填上；真正访问磁盘要等调用表项。

L100–102

```
    table[QEMU_LOGGING] = k_logging;
    table[SET_TIMER] = k_settimer;
    table[READ_FDT] = k_fdt;
```

登记日志、设置定时器和读取设备树信息的底层服务，定时抢占稍后会用到定时器。

L103–104

```
    table[MOVE_CURSOR] = k_cursor;
    table[PRINT] = k_write;
```

登记移动光标与屏幕字符串输出；PRINT走k_write，和直接BIOS字符串服务不完全相同。

L105

```
    table[YIELD] = k_yield;
```

关键连接：YIELD表项指向k_yield，任务主动让出请求由此进入调度器。

L106–108

```
    table[MUTEX_INIT] = k_lock_init;
    table[MUTEX_ACQ] = k_lock_acq;
    table[MUTEX_RELEASE] = k_lock_rel;
```

登记锁初始化、获取和释放，供阶段2任务请求互斥服务。

L109–110

```
    table[REFLUSH] = k_refresh;
}
```

登记屏幕刷新并结束函数。表内是函数地址，不是各函数执行后的返回值。

3. sys_yield：完整分阶段封装

P2:tiny_libc/syscall.c · L18–L25 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+   18 void sys_yield(void)
+   19 {
+   20 #if LAB_STAGE < 3
+   21     call_jmptab(YIELD, 0, 0, 0, 0, 0);
+   22 #else
+   23     invoke_syscall(SYSCALL_YIELD, 0, 0, 0, 0, 0);
+   24 #endif
+   25 }
```

按原始行号逐句理解

L18–19

```
void sys_yield(void)
{
```

对任务提供无参数的统一接口，任务不必知道底层如何切换。

L20–21

```
#if LAB_STAGE < 3
    call_jmptab(YIELD, 0, 0, 0, 0, 0);
```

阶段1/2编译时保留此分支，通过YIELD跳转表项调用内核服务。

L22–23

```
#else
    invoke_syscall(SYSCALL_YIELD, 0, 0, 0, 0, 0);
```

阶段3/4编译时保留另一分支，转成SYSCALL_YIELD系统调用号并执行ecall；不是运行时同时走两条路径。

L24–25

```
#endif
}
```

结束条件编译和函数。调用者以后恢复时从sys_yield之后继续。

4. 验证程序coop1：完整源码

P2:test/validation/coop1.c · L1–L10 · 完整测试程序
相对P1新增的文件；完成版新增

```
+    1 #include <stdio.h>
+    2 #include <unistd.h>
+    3 int main(void)
+    4 {
+    5     for (unsigned i = 0; ; ++i) {
+    6         sys_move_cursor(0, 0);
+    7         printf("[COOP 1] count=%u\n", i);
+    8         sys_yield();
+    9     }
+   10 }
```

按原始行号逐句理解

L1–2

```
#include <stdio.h>
#include <unistd.h>
```

引入printf与sys_yield等声明。其实现随阶段选择跳转表或ecall。

L3–4

```
int main(void)
{
```

任务C入口，由前面讲的crt0调用。

L5

```
    for (unsigned i = 0; ; ++i) {
```

无限计数循环；不靠main返回结束任务。

L6

```
        sys_move_cursor(0, 0);
```

将本任务光标放到指定行，便于与coop2区分。

L7

```
        printf("[COOP 1] count=%u\n", i);
```

打印当前计数，说明任务确实执行到了用户逻辑，而不只是初始化LOAD信息。

L8

```
        sys_yield();
```

主动让出CPU，是阶段1能轮流执行的直接触发点；拿掉后若没有其他阻塞点，就不能依靠阶段1的定时器抢占。

L9–10

```
    }
}
```

每次恢复后进入下一轮，计数增加，再次打印并yield。

把执行过程手算一遍

1. 调用链：coop1 main → sys_yield → call_jmptab(YIELD) → k_yield → do_scheduler → switch_to。
2. 恢复时沿相反的返回链走回coop1，执行循环的下一轮；下一次yield又让出。

应该看到什么，由哪里保证

- Task1中[COOP 1]和[COOP 2]计数都持续更新，证明两条独立调用链能来回恢复。

对应本次验证：stage1（详细范围和结果见末尾）。

> **理解边界：**阶段1/2的普通函数调用没有自动完成U→S特权级切换；不要把它叫ecall系统调用。

Task 2 · 竞争与临界区

09 原子交换、关中断与等待队列各做什么

> **这一步要完成什么**
>
> 既要防止两次竞争同时判定成功，又要保证修改队列与所有权时不会被本核中断打断。

与P1／前一步的关系：P1没有正在工作的互斥锁。P2在锁状态之上增加key、owner与阻塞队列，失败者可以放弃CPU。

输入、输出与运行条件：原子指令完成一次不可分割的状态交换；irq_save保护本核内核数据操作；mutex等待队列记录因资源暂不可用而阻塞的任务。三者不能互相替代。

1. 锁状态和互斥锁结构：完整定义

P2:include/os/lock.h · L35–L52 · 完整类型定义
相对P1新增的文件；课程框架基础上修改／补全

```
+   35 typedef enum {
+   36     UNLOCKED,
+   37     LOCKED,
+   38 } lock_status_t;
+   39
+   40 typedef struct spin_lock
+   41 {
+   42     volatile lock_status_t status;
+   43 } spin_lock_t;
+   44
+   45 typedef struct mutex_lock
+   46 {
+   47     spin_lock_t lock;
+   48     list_head block_queue;
+   49     int key;
+   50     int used;
+   51     pid_t owner;
+   52 } mutex_lock_t;
```

按原始行号逐句理解

L35–38

```
typedef enum {
    UNLOCKED,
    LOCKED,
} lock_status_t;
```

UNLOCKED为0、LOCKED为1，表示底层状态，不是任务pid。

L40–43

```
typedef struct spin_lock
{
    volatile lock_status_t status;
} spin_lock_t;
```

spin_lock_t只包一项status。这里volatile确实修饰状态对象，但让“读旧值并写新值”不可分割的是后面的atomic_swap指令，不能把volatile当原子锁。

L45–46

```
typedef struct mutex_lock
{
```

定义互斥锁对象，是所有使用相同key的任务共同操作的那一把锁。

L47

```
    spin_lock_t lock;
```

底层spin_lock_t记录目前被占用与否。

L48

```
    list_head block_queue;
```

block_queue保存抢锁失败的PCB节点；它与全局ready_queue是不同队列。

L49

```
    int key;
```

key是请求者约定的共享标识，相同key应得到同一个锁槽。

L50

```
    int used;
```

used表示数组槽是否已分配，不能仅用status判断一把锁是否已经创建。

L51–52

```
    pid_t owner;
} mutex_lock_t;
```

owner记录合法持有者pid，释放与恢复时可检查所有权；不是把pid当锁数组下标。

2. atomic_swap：完整32位原子交换

P2:arch/riscv/include/atomic.h · L8–L17 · 完整函数／连续实现块
相对P1新增的文件；课程框架保留的关键依赖

```
+    8 static inline uint32_t atomic_swap(uint32_t val, ptr_t mem_addr)
+    9 {
+   10     uint32_t ret;
+   11     __asm__ __volatile__ (
+   12         "amoswap.w.aqrl %0, %2, %1\n"
+   13         : "=r"(ret), "+A" (*(uint32_t*)mem_addr)
+   14         : "r"(val)
+   15         : "memory");
+   16     return ret;
```

```
+   17 }
```

按原始行号逐句理解

L8–9

```
static inline uint32_t atomic_swap(uint32_t val, ptr_t mem_addr)
{
```

输入新值val和目标内存地址mem_addr，返回值是交换前的旧值。

L10

```
    uint32_t ret;
```

ret接收旧状态，不是交换后的新状态。

L11

```
    __asm__ __volatile__ (
```

volatile修饰内联汇编，要求编译器保留这次有副作用的指令；它与前面修饰函数返回值的错误用法不同。

L12

```
        "amoswap.w.aqrl %0, %2, %1\n"
```

amoswap.w一次把val写入32位内存并取回旧值；aqrl附带获取/释放次序约束，不能拆成普通load加store。

L13

```
        : "=r"(ret), "+A" (*(uint32_t*)mem_addr)
```

=r指定寄存器输出ret，+A把目标内存同时作为读写操作数。

L14

```
        : "r"(val)
```

r约束让输入val放在寄存器，作为要写入的新状态。

L15

```
        : "memory");
```

memory clobber告诉编译器这段汇编影响内存，不能把相关内存访问随意挪过它；硬件原子性仍由amoswap提供。

L16–17

```
    return ret;
}
```

返回旧状态，调用者据此判断自己是否抢到了原来空闲的锁。

3. irq_save / irq_restore：完整本地中断保护

P2:include/os/irq.h · L78–L88 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+   78 static inline uint64_t irq_save(void)
+   79 {
```

```
+   80     uint64_t old;
+   81     __asm__ volatile ("csrrc %0, sstatus, %1" : "=r"(old) : "r"(2UL) : "memory");
+   82     return old;
+   83 }
+   84 static inline void irq_restore(uint64_t old)
+   85 {
+   86     if (old & 2UL)
+   87         __asm__ volatile ("csrs sstatus, %0" :: "r"(2UL) : "memory");
+   88 }
```

按原始行号逐句理解

L78–80

```
static inline uint64_t irq_save(void)
{
    uint64_t old;
```

声明保存并关闭中断的函数，old用于留下进入前的sstatus。

L81

```
    __asm__ volatile ("csrrc %0, sstatus, %1" : "=r"(old) : "r"(2UL) : "memory");
```

csrrc在读出旧sstatus的同时清掉掩码2，也就是SIE位。这样下一步改链表时不会被本核S态中断重入；memory约束也限制编译器重排。

L82–83

```
    return old;
}
```

把旧状态返回给当前这层调用者，供配对恢复使用。

L84–85

```
static inline void irq_restore(uint64_t old)
{
```

恢复接口接收本层保存的旧状态，不读取另一个任务的flags。

L86–88

```
    if (old & 2UL)
        __asm__ volatile ("csrs sstatus, %0" :: "r"(2UL) : "memory");
}
```

只有原先SIE为1才重新置位；原先为0就保持关闭。嵌套调用因而不会由内层过早打开外层关掉的中断。这不是恢复整个sstatus。

4. 自旋基础操作与锁数组初始化：完整函数组

P2:kernel/locking/lock.c · L8–L29 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+    8 void spin_lock_init(spin_lock_t *lock) { lock->status = UNLOCKED; }
+    9 int spin_lock_try_acquire(spin_lock_t *lock)
+   10 {
+   11     return atomic_swap(LOCKED, (ptr_t)&lock->status) == UNLOCKED;
```

```
+   12 }
+   13 void spin_lock_acquire(spin_lock_t *lock)
+   14 {
+   15     while (!spin_lock_try_acquire(lock)) __asm__ volatile ("nop");
+   16 }
+   17 void spin_lock_release(spin_lock_t *lock)
+   18 {
+   19     atomic_swap(UNLOCKED, (ptr_t)&lock->status);
+   20 }
+   21 void init_locks(void)
+   22 {
+   23     for (int i = 0; i < LOCK_NUM; ++i) {
+   24         spin_lock_init(&mlocks[i].lock);
+   25         list_init(&mlocks[i].block_queue);
+   26         mlocks[i].used = 0;
+   27         mlocks[i].owner = 0;
+   28     }
+   29 }
```

按原始行号逐句理解

L8

```
void spin_lock_init(spin_lock_t *lock) { lock->status = UNLOCKED; }
```

初始化底层锁为空闲；普通赋值用于尚未并发使用的初始化阶段。

L9–10

```
int spin_lock_try_acquire(spin_lock_t *lock)
{
```

try接口只尝试一次，返回是否成功。

L11–12

```
    return atomic_swap(LOCKED, (ptr_t)&lock->status) == UNLOCKED;
}
```

原子地写LOCKED并检查旧值。只有旧值是UNLOCKED，才表示本次获得锁；旧值为LOCKED就失败。

L13–16

```
void spin_lock_acquire(spin_lock_t *lock)
{
    while (!spin_lock_try_acquire(lock)) __asm__ volatile ("nop");
}
```

通用自旋获取接口不断尝试，中间执行nop；它会消耗CPU。后面的mutex竞争失败并不走这个忙等循环，而是do_block。

L17–20

```
void spin_lock_release(spin_lock_t *lock)
{
    atomic_swap(UNLOCKED, (ptr_t)&lock->status);
}
```

释放底层锁，用原子交换把状态置UNLOCKED。高层mutex还必须处理owner和等待者。

L21–22

```
void init_locks(void)
{
```

在启用任务运行前初始化整个互斥锁数组。

L23

```
    for (int i = 0; i < LOCK_NUM; ++i) {
```

遍历所有LOCK_NUM个槽，避免只初始化某一个测试使用的锁。

L24

```
        spin_lock_init(&mlocks[i].lock);
```

把每槽底层状态设为空闲。

L25

```
        list_init(&mlocks[i].block_queue);
```

把每槽的等待队列头设为自环。

L26

```
        mlocks[i].used = 0;
```

used清零，表示暂未分配给任何key。

L27

```
        mlocks[i].owner = 0;
```

owner清零，表示没有普通任务持有；本实现普通任务pid从1开始。

L28–29

```
    }
}
```

完成循环与初始化函数；后续创建锁只分配槽，不重新破坏已有等待队列。

把执行过程手算一遍

1. 用普通load看status=0，再普通store写1，中间可能发生竞争；atomic_swap把这两件事合成一条原子操作。
2. 拿到锁仍不能在修改block_queue一半时允许本核调度重入，所以高层操作保存并关闭中断。当前设计针对单核，不是完整SMP锁实现。

应该看到什么，由哪里保证

- 两个任务不能同时把同一把锁当成空闲。持锁任务yield之后，等待者应阻塞，而不是一直占着CPU重试。

对应本次验证：stage2 / stage4（详细范围和结果见末尾）。

> **理解边界：**volatile不会自动实现原子性，也不会把普通链表操作变成线程安全。 关本核中断不能阻止其他CPU访问同一数据，所以不能据此宣称支持多核。

Task 2 · 共享同一把锁

10 同key返回同槽，索引必须有效

> **这一步要完成什么**
>
> 让两个独立任务通过约定的key找到同一互斥锁，并拒绝无效槽索引。

与P1／前一步的关系：如果每次init都新建一把锁，即使两个任务都写了acquire，也无法互斥。P2在锁数组中按key复用对象。

输入、输出与运行条件：输入是整数key；输出是锁数组下标，数组已满时返回-1。它不是直接返回锁对象地址。

1. do_mutex_lock_init和get_lock：完整实现

P2:kernel/locking/lock.c · L30–L52 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+   30 int do_mutex_lock_init(int key)
+   31 {
+   32     uint64_t flags = irq_save();
+   33     int free_slot = -1;
+   34     for (int i = 0; i < LOCK_NUM; ++i) {
+   35         if (mlocks[i].used && mlocks[i].key == key) {
+   36             irq_restore(flags);
+   37             return i;
+   38         }
+   39         if (!mlocks[i].used && free_slot < 0) free_slot = i;
+   40     }
+   41     if (free_slot >= 0) {
+   42         mlocks[free_slot].used = 1;
+   43         mlocks[free_slot].key = key;
+   44     }
+   45     irq_restore(flags);
+   46     return free_slot;
+   47 }
+   48 static mutex_lock_t *get_lock(int id)
+   49 {
+   50     assert(id >= 0 && id < LOCK_NUM && mlocks[id].used);
+   51     return &mlocks[id];
+   52 }
```

按原始行号逐句理解

L30–31

```
int do_mutex_lock_init(int key)
{
```

创建或取得同key的锁，key只是共享标识，允许任务各自调用这个接口。

L32

```
    uint64_t flags = irq_save();
```

保存并关闭本地中断，保护“查已有槽并分配空槽”的整个过程。

L33

```
    int free_slot = -1;
```

free_slot初始-1，表示还没发现可用空位。

L34

```
    for (int i = 0; i < LOCK_NUM; ++i) {
```

逐槽搜索，不能只看到一个空位就立即退出，否则后面可能还有同key的已用槽。

L35

```
        if (mlocks[i].used && mlocks[i].key == key) {
```

若该槽已用且key相等，说明共享对象已经存在。

L36–37

```
            irq_restore(flags);
            return i;
```

恢复本层中断状态，返回原槽下标，两个请求者于是会操作同一lock和block_queue。

L38

```
        }
```

结束命中分支。

L39

```
        if (!mlocks[i].used && free_slot < 0) free_slot = i;
```

只记住第一个空槽，同时继续搜完整数组，优先保证同key复用。

L40

```
    }
```

搜索循环结束。

L41

```
    if (free_slot >= 0) {
```

确实找到空位才进行分配；数组全满时保持-1。

L42–43

```
        mlocks[free_slot].used = 1;
        mlocks[free_slot].key = key;
```

标记使用并记录key。底层状态和空等待队列已经由init_locks准备好。

L44

```
    }
```

分配分支结束。

L45–47

```
    irq_restore(flags);
    return free_slot;
}
```

恢复中断并返回槽号，或者返回-1；调用者必须检查创建结果。

L48–49

```
static mutex_lock_t *get_lock(int id)
{
```

get_lock是内部辅助函数，把外部整数id转换成可操作的锁对象指针。

L50

```
    assert(id >= 0 && id < LOCK_NUM && mlocks[id].used);
```

检查id非负、小于LOCK_NUM且该槽已分配。&&短路保证无效下标不会先被用于读取mlocks[id]。

L51–52

```
    return &mlocks[id];
}
```

检查通过后返回该槽地址。该断言是当前内核的错误处理方式，不是返回一个用户态错误码。

把执行过程手算一遍

1. A调用init(42)先得到id=0；B调用init(42)仍得到0；C调用init(99)才可能取得另一个槽。
2. mutex1中的assert(id == sys_mutex_init(42))直接验证重复初始化同key不会偷偷创建新对象。

应该看到什么，由哪里保证

- 两个任务的获取/释放事件围绕同一把锁发生；不同key应拥有独立的等待队列和owner。

对应本次验证：stage2 / stage4（详细范围和结果见末尾）。

> **理解边界：**key不是数组下标，不能把用户输入key直接用作mlocks[key]。 数组容量有限；返回-1不是有效锁。

Task 2 · 阻塞与移交

11 拿不到锁就睡，释放时把所有权交给等待者

> **这一步要完成什么**
>
> 把BLOCKED状态、等待队列、调度和FIFO所有权移交接成完整协议。

与P1／前一步的关系：P1只有一条同步调用链。P2等待资源的任务可暂时离开CPU，由其他任务继续运行并最终释放资源。

输入、输出与运行条件：do_block的node必须属于当前普通任务且已脱链；do_unblock接收等待队列中的节点。mutex的获取与释放在关本地中断的保护下操作。

1. do_block与do_unblock：完整状态／队列迁移

P2:kernel/sched/sched.c · L51–L71 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+   51 void do_block(list_node_t *node, list_head *queue)
+   52 {
+   53     uint64_t flags = irq_save();
+   54     assert(node == &current_running->list && current_running != &pid0_pcb);
+   55     assert(list_empty(node));
+   56     current_running->status = TASK_BLOCKED;
+   57     list_add_tail(node, queue);
+   58     do_scheduler();
+   59     irq_restore(flags);
+   60 }
+   61
+   62 void do_unblock(list_node_t *node)
+   63 {
+   64     uint64_t flags = irq_save();
+   65     pcb_t *task = list_entry(node, pcb_t, list);
+   66     assert(task->status == TASK_BLOCKED);
+   67     list_del(node);
+   68     task->status = TASK_READY;
+   69     list_add_tail(node, &ready_queue);
+   70     irq_restore(flags);
+   71 }
```

按原始行号逐句理解

L51–53

```
void do_block(list_node_t *node, list_head *queue)
{
    uint64_t flags = irq_save();
```

阻塞当前任务，先保存并关闭中断，保护状态与队列的联合更新。

L54

```
    assert(node == &current_running->list && current_running != &pid0_pcb);
```

确认node恰好是当前任务的list，且当前任务不是pid0；idle不能作为普通等待者被堵死。

L55

```
    assert(list_empty(node));
```

list_empty(node)检查节点处于自环脱链状态。不能把仍在ready_queue里的同一节点再挂到锁队列。

L56

```
    current_running->status = TASK_BLOCKED;
```

先标记BLOCKED，告诉do_scheduler不要把它重新排回就绪队尾。

L57

```
    list_add_tail(node, queue);
```

再把节点尾插进传入的等待队列；队列可属于锁，也可为sleep_queue。

L58

```
    do_scheduler();
```

运行调度器，当前任务停止继续执行这里的后续语句，转由其他任务运行。

L59–60

```
    irq_restore(flags);
}
```

将来被唤醒、重新调度回来后才恢复本层中断状态并返回。do_block的返回本身意味着它已走过一次暂停和恢复。

L62–64

```
void do_unblock(list_node_t *node)
{
    uint64_t flags = irq_save();
```

唤醒操作同样保存并关闭本地中断。

L65

```
    pcb_t *task = list_entry(node, pcb_t, list);
```

从内嵌节点地址找回所属PCB。

L66

```
    assert(task->status == TASK_BLOCKED);
```

确认任务原来确实在BLOCKED状态，避免随意唤醒一个正在运行的任务。

L67

```
    list_del(node);
```

先从原等待队列脱链。

L68

```
    task->status = TASK_READY;
```

改为READY，表示具备运行资格，但还没有立刻抢到CPU。

L69

```
    list_add_tail(node, &ready_queue);
```

追加到就绪队尾，等待调度器以后选中。

L70–71

```
    irq_restore(flags);
}
```

恢复中断并返回。唤醒不是直接调用这个任务的入口，更不是重新从main开始。

2. 互斥锁获取与释放：完整函数组

P2:kernel/locking/lock.c · L53–L81 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+   53 void do_mutex_lock_acquire(int id)
+   54 {
+   55     uint64_t flags = irq_save();
+   56     mutex_lock_t *lock = get_lock(id);
+   57     assert(lock->owner != current_running->pid); /* non-recursive */
+   58     if (spin_lock_try_acquire(&lock->lock)) {
+   59         lock->owner = current_running->pid;
+   60     } else {
+   61         do_block(&current_running->list, &lock->block_queue);
+   62         /* release handed ownership to us; no stealing by a new arrival. */
+   63         assert(lock->owner == current_running->pid);
+   64     }
+   65     irq_restore(flags);
+   66 }
+   67 void do_mutex_lock_release(int id)
+   68 {
+   69     uint64_t flags = irq_save();
+   70     mutex_lock_t *lock = get_lock(id);
+   71     assert(lock->owner == current_running->pid);
+   72     if (!list_empty(&lock->block_queue)) {
+   73         list_node_t *node = lock->block_queue.next;
+   74         lock->owner = list_entry(node, pcb_t, list)->pid;
+   75         do_unblock(node); /* lock remains LOCKED until this owner releases */
+   76     } else {
+   77         lock->owner = 0;
+   78         spin_lock_release(&lock->lock);
+   79     }
+   80     irq_restore(flags);
+   81 }
```

按原始行号逐句理解

L53–54

```
void do_mutex_lock_acquire(int id)
{
```

接收锁id，准备获取共享互斥锁。

L55

```
    uint64_t flags = irq_save();
```

保护本次检查、占用或阻塞过程，不能在“看见被占用”和“挂等待队列”之间被另一条路径插入。

L56

```
    mutex_lock_t *lock = get_lock(id);
```

校验id并取得锁对象。

L57

```
    assert(lock->owner != current_running->pid); /* non-recursive */
```

禁止持有者递归获取同一把锁，避免自己等待自己；这是非递归锁约定。

L58

```
    if (spin_lock_try_acquire(&lock->lock)) {
```

只进行一次原子尝试，成功与失败分别处理。

L59

```
        lock->owner = current_running->pid;
```

成功时记录当前任务pid为owner。

L60–61

```
    } else {
        do_block(&current_running->list, &lock->block_queue);
```

失败时调用do_block进入这把锁的block_queue，放弃CPU；不是在用户栈上忙等直到锁变空。

L62–63

```
        /* release handed ownership to us; no stealing by a new arrival. */
        assert(lock->owner == current_running->pid);
```

能从do_block返回，表示以后已被唤醒并调度回来。释放者已经把owner移交给它，所以这里检查owner，不再重新抢一次锁。

L64–66

```
    }
    irq_restore(flags);
}
```

结束分支、恢复原中断状态，再返回给调用者进入临界区。

L67–68

```
void do_mutex_lock_release(int id)
{
```

接收id，准备释放当前任务所持的锁。

L69–70

```
    uint64_t flags = irq_save();
    mutex_lock_t *lock = get_lock(id);
```

保护释放过程并校验id。

L71

```
    assert(lock->owner == current_running->pid);
```

只有当前owner可以释放，防止任务A把任务B的锁解掉。

L72

```
    if (!list_empty(&lock->block_queue)) {
```

如果还有等待者，采取所有权移交路径。

L73

```
        list_node_t *node = lock->block_queue.next;
```

取等待队列第一个节点，实现FIFO优先。

L74

```
        lock->owner = list_entry(node, pcb_t, list)->pid;
```

先把owner改为该等待者pid；下次它从acquire恢复时，断言应成立。

L75

```
        do_unblock(node); /* lock remains LOCKED until this owner releases */
```

把该任务从等待队列移到ready_queue。底层status保持LOCKED，后来新到的任务不会趁等待者尚未运行而抢走锁。

L76

```
    } else {
```

没有等待者才走真正解锁分支。

L77–78

```
        lock->owner = 0;
        spin_lock_release(&lock->lock);
```

清owner，并把底层状态设UNLOCKED，未来的新请求才能通过原子try取得。

L79–81

```
    }
    irq_restore(flags);
}
```

释放分支结束，恢复原中断状态并返回。这里并不保证被唤醒任务马上运行，实际切换仍由调度触发。

3. mutex1：持锁时故意yield的完整测试

P2:test/validation/mutex1.c · L1–L18 · 完整测试程序
相对P1新增的文件；完成版新增

```
+    1 #include <stdio.h>
+    2 #include <unistd.h>
+    3 #include <assert.h>
+    4 int main(void)
+    5 {
+    6     int id = sys_mutex_init(42);
+    7     assert(id >= 0 && id == sys_mutex_init(42));
```

```
+    8     for (unsigned n = 0;; ++n) {
+    9         sys_mutex_acquire(id);
+   10         sys_move_cursor(0, 0);
+   11         printf("[MUTEX 1] acquired %u\n", n);
+   12         sys_yield(); /* force the peer to contend while we own the lock */
+   13         sys_move_cursor(0, 0);
+   14         printf("[MUTEX 1] release %u\n", n);
+   15         sys_mutex_release(id);
+   16         sys_yield();
+   17     }
+   18 }
```

按原始行号逐句理解

L1–3

```
#include <stdio.h>
#include <unistd.h>
#include <assert.h>
```

引入输出、系统接口与断言；该程序是验证代码，不是锁本体。

L4–5

```
int main(void)
{
```

由任务crt0进入C main。

L6

```
    int id = sys_mutex_init(42);
```

请求key=42对应的锁槽。

L7

```
    assert(id >= 0 && id == sys_mutex_init(42));
```

检查创建成功，并检查第二次同key请求返回同id。

L8

```
    for (unsigned n = 0;; ++n) {
```

持续进行多轮获取与释放，避免只验证一次幸运执行。

L9

```
        sys_mutex_acquire(id);
```

获取成功后才能继续下面的临界区操作；失败者应在内核阻塞。

L10–11

```
        sys_move_cursor(0, 0);
        printf("[MUTEX 1] acquired %u\n", n);
```

移动光标并打印acquired，作为进入临界区的标记。

L12

```
        sys_yield(); /* force the peer to contend while we own the lock */
```

关键：锁仍在手里时主动yield，让对方真的有机会来竞争，从而覆盖“失败后阻塞”的路径。

L13–14

```
        sys_move_cursor(0, 0);
        printf("[MUTEX 1] release %u\n", n);
```

本任务恢复后打印即将释放的标记。

L15

```
        sys_mutex_release(id);
```

释放并可能把所有权交给等待者。

L16

```
        sys_yield();
```

再主动yield，便于被唤醒的任务取得CPU；不能把这次yield误当成解锁本身。

L17–18

```
    }
}
```

进入下一轮，继续制造竞争并观察两任务都能前进。

把执行过程手算一遍

1. A持锁并yield → B尝试失败，标BLOCKED并入锁队列 → 调度回A。
2. A释放：owner先交给B，B变READY，底层仍LOCKED → 新任务C不能抢走 → B恢复到原do_block返回处，检查owner后进入临界区。
3. 释放时若无等待者，才将owner置0、底层UNLOCKED。两条释放路径不能合并成“先解锁再唤醒”。

应该看到什么，由哪里保证

- Task2双方反复出现acquired/release；Task4探针还覆盖持锁sleep、等待者阻塞和醒后移交。

对应本次验证：stage2 / stage4（详细范围和结果见末尾）。

> **理解边界：**先把锁置空再唤醒，会允许新到达任务插队抢走资源，使被唤醒者的owner断言失败。 do_unblock后任务只进入READY；当场运行与否由下一次调度决定。

Task 3 · 用户发起请求

12 把服务号和参数放入寄存器，再执行ecall

> **这一步要完成什么**
>
> 让U态任务通过受控的异常入口请求内核服务，并把返回值带回调用者。

与P1／前一步的关系：P1与P2阶段1/2主要用普通函数指针调用。Task3开始，任务运行在U态，系统服务接口改为ecall；底层内核调用BIOS是另外一层。

输入、输出与运行条件：输入number与p0到p4；调用前a7=服务号、a0到a4=参数；返回时a0=结果。用户现场会由内核异常入口保护。

1. invoke_syscall：完整用户态汇编封装

P2:tiny_libc/syscall.c · L6–L17 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+    6 static long invoke_syscall(long number, long p0, long p1, long p2, long p3, long p4)
+    7 {
+    8     register long a0 asm("a0") = p0;
+    9     register long a1 asm("a1") = p1;
+   10     register long a2 asm("a2") = p2;
+   11     register long a3 asm("a3") = p3;
+   12     register long a4 asm("a4") = p4;
+   13     register long a7 asm("a7") = number;
+   14     asm volatile ("ecall" : "+r"(a0) : "r"(a1), "r"(a2), "r"(a3),
+   15                   "r"(a4), "r"(a7) : "memory");
+   16     return a0;
+   17 }
```

按原始行号逐句理解

L6–7

```
static long invoke_syscall(long number, long p0, long p1, long p2, long p3, long p4)
{
```

统一的内部系统调用封装。它接收服务号和五个long参数，给多个sys_*接口复用。

L8

```
    register long a0 asm("a0") = p0;
```

把第一个参数绑定到硬件寄存器a0。这个变量也承担返回值，因此稍后在汇编约束中是读写操作数。

L9–12

```
    register long a1 asm("a1") = p1;
    register long a2 asm("a2") = p2;
    register long a3 asm("a3") = p3;
    register long a4 asm("a4") = p4;
```

将其余四个参数分别绑定到a1、a2、a3、a4，顺序必须与内核读取regs槽位的顺序一致。

L13

```
    register long a7 asm("a7") = number;
```

把服务号绑定到a7。此处号码属于内核系统调用表，不应与BIOS服务号混为一谈。

L14–15

```
    asm volatile ("ecall" : "+r"(a0) : "r"(a1), "r"(a2), "r"(a3),
                  "r"(a4), "r"(a7) : "memory");
```

执行ecall，CPU陷入S态异常处理。+r表示a0既是输入也是输出；其余寄存器为输入；memory clobber限制跨调用的内存重排。asm volatile要求保留这次有副作用的汇编。

L16

```
    return a0;
```

内核通过sret回来后，从a0取回结果作为C函数返回值；不是在用户代码里直接算出结果。

L17

```
}
```

函数结束。运行中可能经历调度，但最终仍回到这个用户调用链。

2. sys_write：完整分阶段服务接口

P2:tiny_libc/syscall.c · L34–L41 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+   34 void sys_write(char *buff)
+   35 {
+   36 #if LAB_STAGE < 3
+   37     call_jmptab(PRINT, (long)buff, 0, 0, 0, 0);
+   38 #else
+   39     invoke_syscall(SYSCALL_WRITE, (long)buff, 0, 0, 0, 0);
+   40 #endif
+   41 }
```

按原始行号逐句理解

L34–35

```
void sys_write(char *buff)
{
```

向任务提供字符串输出接口，buff是任务地址空间中的字符指针。

L36–37

```
#if LAB_STAGE < 3
    call_jmptab(PRINT, (long)buff, 0, 0, 0, 0);
```

阶段1/2直接经PRINT跳转表项调用内核服务，任务仍在S态。

L38–39

```
#else
    invoke_syscall(SYSCALL_WRITE, (long)buff, 0, 0, 0, 0);
```

阶段3/4把指针转成一个long参数，使用SYSCALL_WRITE发起ecall。指针类型转换不会复制字符串内容。

L40–41

```
#endif
}
```

条件编译结束。对任务而言仍调用sys_write，服务机制的变化由封装层承担。

把执行过程手算一遍

1. 用户写sys_write("hello") → 参数指针进a0，号码进a7 → ecall。接下来CPU自动改变控制流，跳转目标由stvec决定。
2. 与前面BIOS包装对照：call_bios最后普通调用固件入口；invoke_syscall这里发起的是U态到内核的异常，不是把ecall强转成函数指针。

应该看到什么，由哪里保证

- Task3的sleep与coop_timer能在U态运行并输出，说明请求能往返；未知调用号与错误字符串指针由更强探针检查。

对应本次验证：stage3 / stage4（详细范围和结果见末尾）。

> **理解边界：**ecall本身不认识“打印”和“睡眠”；功能含义来自a7号码与内核分发表。 用户参数是数值/地址，内核在解引用指针前仍需检查范围；相关完整k_write实现放在后面的输出附录。

Task 3 · 异常进入内核

13 先找可靠的PCB，再选栈和保存现场

这一步要完成什么

使ecall和后续时钟中断进入统一入口，并保住所有用户通用寄存器和必要CSR。

与P1／前一步的关系：P1没有正在工作的U态异常往返链。P2必须面对“被打断时用户可在任意指令、任意寄存器状态”的情况，不能只保存s0到s11。

输入、输出与运行条件：硬件进入S态时会记录sepc/scause/stval并更新sstatus，通用寄存器仍要软件保存。正常边界sscratch指当前PCB，即使用户tp被改成其他值也能定位内核栈。

1. setup_exception：完整异常入口注册

P2:arch/riscv/kernel/trap.S · L1–L11 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+    1 #include <asm.h>
+    2 #include <csr.h>
+    3 ENTRY(setup_exception)
+    4     la t0, exception_handler_entry
+    5     csrw CSR_STVEC, t0
+    6     csrw CSR_SSCRATCH, tp
+    7     li t0, 2 /* permit user rdtime when M-mode also permits it */
+    8     csrs CSR_SCOUNTEREN, t0
+    9     /* Enable only after all tables, queues and the timer are ready. */
+   10     ret
+   11 ENDPROC(setup_exception)
```

按原始行号逐句理解

L1–2

```
#include <asm.h>
#include <csr.h>
```

引入汇编宏和CSR符号，避免把寄存器编号散落成裸数字。

L3

```
ENTRY(setup_exception)
```

定义供init_exception调用的setup_exception。

L4

```
    la t0, exception_handler_entry
```

取得统一汇编异常入口exception_handler_entry的地址。

L5

```
    csrw CSR_STVEC, t0
```

把入口写入stvec，决定S态异常/中断到达时从哪里执行；这不是现在就触发异常。

L6

```
    csrw CSR_SSCRATCH, tp
```

把当前PCB地址存入sscratch，作为进入异常时的可靠锚点。

L7–8

```
    li t0, 2 /* permit user rdtime when M-mode also permits it */
    csrs CSR_SCOUNTEREN, t0
```

允许用户读取time计数器；同时仍需M态提供对应许可。它不会启用定时中断。

L9

```
    /* Enable only after all tables, queues and the timer are ready. */
```

注释强调此时只安装入口，开中断要等队列、表和定时器准备好。

L10–11

```
    ret
ENDPROC(setup_exception)
```

返回初始化调用者，并结束符号。

2. SAVE_CONTEXT：完整宏，不省略保存指令

P2:arch/riscv/kernel/entry.S · L8–L68 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+    8 .macro SAVE_CONTEXT
+    9     csrrw tp, CSR_SSCRATCH, tp
+   10     sd t0, PCB_TRAP_T0(tp)
+   11     csrr t0, CSR_SSTATUS
+   12     andi t0, t0, SR_SPP
+   13     bnez t0, 1f
+   14     sd sp, PCB_USER_SP(tp)
+   15     ld sp, PCB_KERNEL_SP(tp)
+   16 1:  addi sp, sp, -OFFSET_SIZE
+   17     sd zero, 0(sp)
+   18     sd ra, 8(sp)
+   19     sd gp, 24(sp)
+   20     sd t1, 48(sp)
+   21     sd t2, 56(sp)
+   22     sd s0, 64(sp)
+   23     sd s1, 72(sp)
+   24     sd a0, 80(sp)
+   25     sd a1, 88(sp)
+   26     sd a2, 96(sp)
+   27     sd a3, 104(sp)
+   28     sd a4, 112(sp)
+   29     sd a5, 120(sp)
+   30     sd a6, 128(sp)
```

```
+   31     sd a7, 136(sp)
+   32     sd s2, 144(sp)
+   33     sd s3, 152(sp)
+   34     sd s4, 160(sp)
+   35     sd s5, 168(sp)
+   36     sd s6, 176(sp)
+   37     sd s7, 184(sp)
+   38     sd s8, 192(sp)
+   39     sd s9, 200(sp)
+   40     sd s10, 208(sp)
+   41     sd s11, 216(sp)
+   42     sd t3, 224(sp)
+   43     sd t4, 232(sp)
+   44     sd t5, 240(sp)
+   45     sd t6, 248(sp)
+   46     ld t0, PCB_TRAP_T0(tp)
+   47     sd t0, OFFSET_REG_T0(sp)
+   48     csrr t0, CSR_SSCRATCH
+   49     sd t0, OFFSET_REG_TP(sp)
+   50     csrw CSR_SSCRATCH, tp
+   51     csrr t0, CSR_SSTATUS
+   52     sd t0, OFFSET_REG_SSTATUS(sp)
+   53     andi t0, t0, SR_SPP
+   54     beqz t0, 2f
+   55     addi t0, sp, OFFSET_SIZE
+   56     j 3f
+   57 2:  ld t0, PCB_USER_SP(tp)
+   58 3:  sd t0, OFFSET_REG_SP(sp)
+   59     csrr t0, CSR_SEPC
+   60     sd t0, OFFSET_REG_SEPC(sp)
+   61     csrr t0, CSR_STVAL
+   62     sd t0, OFFSET_REG_SBADADDR(sp)
+   63     csrr t0, CSR_SCAUSE
+   64     sd t0, OFFSET_REG_SCAUSE(sp)
+   65     /* No FPU instructions in kernel or user binaries. */
+   66     li t0, SR_SUM | SR_FS
+   67     csrc CSR_SSTATUS, t0
+   68 .endm
```

按原始行号逐句理解

L8

```
.macro SAVE_CONTEXT
```

定义宏；使用时汇编器展开指令，没有一次独立C调用自动替你保存现场。

L9

```
    csrrw tp, CSR_SSCRATCH, tp
```

交换tp与sscratch：tp取得PCB地址，sscratch暂存原用户tp。这个短窗口内仍关中断；稍后会把sscratch恢复成PCB。

L10

```
    sd t0, PCB_TRAP_T0(tp)
```

先把原t0存进PCB的trap_t0槽。后面要借用t0检查状态，必须先保护原值。

L11–13

```
    csrr t0, CSR_SSTATUS
    andi t0, t0, SR_SPP
    bnez t0, 1f
```

读取硬件记录的sstatus并检查SPP，判断异常来自S态还是U态。SPP=1时跳过用户栈切换。

L14–15

```
    sd sp, PCB_USER_SP(tp)
    ld sp, PCB_KERNEL_SP(tp)
```

来自U态时，先把原用户sp存进PCB.user_sp，再换成该任务保存的内核sp；从现在起才在可信内核栈上压帧。

L16

```
1:  addi sp, sp, -OFFSET_SIZE
```

在所选内核栈上预留OFFSET_SIZE=288字节。来自S态时沿当前S态栈继续压帧，不用错误地切回一份旧用户栈。

L17

```
    sd zero, 0(sp)
```

给x0槽写0，形成完整一致的32槽通用寄存器数组。

L18–19

```
    sd ra, 8(sp)
    sd gp, 24(sp)
```

保存ra与gp。ra可能是用户某层函数返回地址，gp可能含用户数据基址，两者都不能被内核覆盖丢失。

L20–23

```
    sd t1, 48(sp)
    sd t2, 56(sp)
    sd s0, 64(sp)
    sd s1, 72(sp)
```

保存t1、t2、s0、s1，各8字节；t0和tp有特殊保存路径，不能在这里把已借用的值当原值存下。

L24–31 完整保存a0到a7。ecall参数与号码就在这里；被异步打断的普通计算也可能正使用这些寄存器。

L32–41 保存s2到s11，固定偏移从144到216，每项递增8。完整代码保留每条sd，讲解合并只是因为保存规则相同。

L42–45

```
    sd t3, 224(sp)
    sd t4, 232(sp)
    sd t5, 240(sp)
    sd t6, 248(sp)
```

保存t3到t6。抢占可能发生在临时计算尚未完成时，所以临时寄存器也必须保住。

L46–47

```
    ld t0, PCB_TRAP_T0(tp)
    sd t0, OFFSET_REG_T0(sp)
```

从PCB临时槽取回原t0，写入异常帧的x5槽；这里存的才是用户原值。

L48–49

```
    csrr t0, CSR_SSCRATCH
    sd t0, OFFSET_REG_TP(sp)
```

从sscratch取出先前暂存的原tp，写到异常帧x4槽。即使用户tp不指向PCB，也能在返回时恢复。

L50

```
    csrw CSR_SSCRATCH, tp
```

把sscratch重新设为当前PCB，恢复正常边界约定。

L51–52

```
    csrr t0, CSR_SSTATUS
    sd t0, OFFSET_REG_SSTATUS(sp)
```

保存sstatus，供返回时恢复原先的模式和中断相关状态。

L53–58 按SPP恢复“陷入前sp”应是什么：S态来源用压帧前的sp，U态来源用PCB里保存的用户sp；统一存到x2槽。

L59–60

```
    csrr t0, CSR_SEPC
    sd t0, OFFSET_REG_SEPC(sp)
```

保存硬件记录的sepc，它指向发生异常或中断时的相关指令位置。

L61–62

```
    csrr t0, CSR_STVAL
    sd t0, OFFSET_REG_SBADADDR(sp)
```

把stval保存到结构体中名为sbadaddr的成员，供错误诊断使用。

L63–64

```
    csrr t0, CSR_SCAUSE
    sd t0, OFFSET_REG_SCAUSE(sp)
```

保存scause，后续据此区分系统调用、时钟中断或其他异常。

L65–67

```
    /* No FPU instructions in kernel or user binaries. */
    li t0, SR_SUM | SR_FS
    csrc CSR_SSTATUS, t0
```

清SUM/FS相关状态，维持当前内核约定；目标构建使用无浮点寄存器ABI，不在这里保存FPU现场。不能据此宣称支持任意浮点程序。

L68

```
.endm
```

宏结束，此时sp指向完整异常帧，tp指当前PCB，可安全转给C处理。

3. exception_handler_entry：完整汇编到C入口

P2:arch/riscv/kernel/entry.S · L171–L178 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+ 171 ENTRY(exception_handler_entry)
+ 172     SAVE_CONTEXT
+ 173     mv a0, sp
+ 174     ld a1, OFFSET_REG_SBADADDR(sp)
+ 175     ld a2, OFFSET_REG_SCAUSE(sp)
+ 176     call interrupt_helper
+ 177     j ret_from_exception
+ 178 ENDPROC(exception_handler_entry)
```

按原始行号逐句理解

L171

```
ENTRY(exception_handler_entry)
```

统一异常入口符号，stvec已被设置为它。

L172

```
    SAVE_CONTEXT
```

展开上一整段宏，把原CPU状态落到内核栈。

L173

```
    mv a0, sp
```

把当前sp，也就是regs_context_t帧地址，作为C函数第一个参数a0。

L174–175

```
    ld a1, OFFSET_REG_SBADADDR(sp)
    ld a2, OFFSET_REG_SCAUSE(sp)
```

从已保存帧中取得stval和scause，放进a1/a2。

L176

```
    call interrupt_helper
```

普通call进入interrupt_helper。用户原ra已经在帧里，这次C调用可以使用当前ra。

L177

```
    j ret_from_exception
```

C处理返回后跳到统一返回路径；不能直接ret回用户，因为还需恢复帧并执行sret。

L178

```
ENDPROC(exception_handler_entry)
```

结束入口符号。

4. interrupt_helper：完整异常／中断分流

P2:kernel/irq/irq.c · L15–L25 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+  15 void interrupt_helper(regs_context_t *regs, uint64_t stval, uint64_t scause)
+  16 {
```

```
+   17     uint64_t code = scause & ~SCAUSE_IRQ_FLAG;
+   18     if (scause & SCAUSE_IRQ_FLAG) {
+   19         if (code < IRQC_COUNT) irq_table[code](regs, stval, scause);
+   20         else handle_other(regs, stval, scause);
+   21     } else {
+   22         if (code < EXCC_COUNT) exc_table[code](regs, stval, scause);
+   23         else handle_other(regs, stval, scause);
+   24     }
+   25 }
```

按原始行号逐句理解

L15–16

```
void interrupt_helper(regs_context_t *regs, uint64_t stval, uint64_t scause)
{
```

输入为已保存的寄存器帧、stval和scause，函数负责选择处理者。

L17

```
    uint64_t code = scause & ~SCAUSE_IRQ_FLAG;
```

去掉scause中的中断标志位，得到原因编号。

L18

```
    if (scause & SCAUSE_IRQ_FLAG) {
```

若最高标志位为1，表示中断，走irq_table。

L19–20

```
        if (code < IRQC_COUNT) irq_table[code](regs, stval, scause);
        else handle_other(regs, stval, scause);
```

编号在范围内才索引中断表，否则进入错误诊断，避免越界取函数地址。

L21

```
    } else {
```

否则是同步异常，如U态ecall。

L22–23

```
        if (code < EXCC_COUNT) exc_table[code](regs, stval, scause);
        else handle_other(regs, stval, scause);
```

按异常编号调用exc_table的处理者，同样先检查范围。U态ecall对应EXCC_SYSCALL=8。

L24–25

```
    }
}
```

分流结束，返回汇编入口。具体服务号仍需系统调用处理器从a7槽读取，不能把scause当服务号。

把执行过程手算一遍

1. 用户原tp=任意测试值X：交换后tp=PCB、sscratch=X；先保存原t0，再选内核栈；把X写入异常帧后sscratch恢复PCB。
2. 这条路径对用户寄存器值保持正确，对idle的S态时钟中断也能选择当前S栈；它们通过SPP区别。

应该看到什么，由哪里保证

- Task4寄存器探针主动改变tp、临时和参数寄存器，经过异常/抢占后检查其值；完整保存与完整恢复共同保证通过。

对应本次验证：stage3 / stage4 / idle（详细范围和结果见末尾）。

> **理解边界：**仅检查一个简单计数器不足以证明全部寄存器正确。 先修改t0再保存会丢原值；盲信用户tp就是PCB可能在异常第一步就写错内存。

Task 3 · 分发系统服务

14 根据a7查表，结果放回a0，ecall的PC加4

> **这一步要完成什么**
>
> 建立内核系统调用表，并让同步异常真正到达yield、输出、睡眠和锁等服务。

与P1／前一步的关系：P1的jmptab在固定内存保存服务地址。P2这里的syscall[]是异常处理器使用的另一张表；两者用途和号码空间不同。

输入、输出与运行条件：输入是regs帧，a7保存在regs[17]；处理后修改regs[10]作为返回值，必要时服务可以阻塞并发生任务切换。

1. sleep服务包装：完整函数

P2:init/main.c · L62–L63 · 完整函数／连续实现块
相对P1修改的文件；课程框架基础上修改／补全

```
+   62 static long k_sleep(long s, long b, long c, long d, long e)
+   63 { do_sleep((uint32_t)s); return 0; }
```

按原始行号逐句理解

L62

```
static long k_sleep(long s, long b, long c, long d, long e)
```

统一五个long参数的内核服务签名；第一个s携带请求秒数，其余参数此服务不用。

L63

```
{ do_sleep((uint32_t)s); return 0; }
```

把s转成uint32_t后交给do_sleep。若阻塞，将来被恢复才继续返回0给系统调用处理器。

2. init_syscall：完整服务表初始化

P2:init/main.c · L173–L186 · 完整函数／连续实现块
相对P1修改的文件；课程框架基础上修改／补全

```
+  173 static void init_syscall(void)
+  174 {
+  175     bzero(syscall, sizeof(long (*)()) * NUM_SYSCALLS);
+  176     syscall[SYSCALL_YIELD] = k_yield;
+  177     syscall[SYSCALL_WRITE] = k_write;
+  178     syscall[SYSCALL_CURSOR] = k_cursor;
+  179     syscall[SYSCALL_REFLUSH] = k_refresh;
+  180     syscall[SYSCALL_SLEEP] = k_sleep;
+  181     syscall[SYSCALL_LOCK_INIT] = k_lock_init;
+  182     syscall[SYSCALL_LOCK_ACQ] = k_lock_acq;
```

```
+ 183     syscall[SYSCALL_LOCK_RELEASE] = k_lock_rel;
+ 184     syscall[SYSCALL_GET_TIMEBASE] = k_timebase;
+ 185     syscall[SYSCALL_GET_TICK] = k_ticks;
+ 186 }
```

按原始行号逐句理解

L173–174

```
static void init_syscall(void)
{
```

在开放异常与任务运行前建立内核分发表。

L175

```
    bzero(syscall, sizeof(long (*)()) * NUM_SYSCALLS);
```

将全部函数指针槽清零，未注册号码可以被明确识别。这里是当前RV64环境使用的零指针表示。

L176

```
    syscall[SYSCALL_YIELD] = k_yield;
```

YIELD号码映射到k_yield，再由它调用调度器。

L177

```
    syscall[SYSCALL_WRITE] = k_write;
```

WRITE号码映射到带字符串范围检查的k_write。

L178–179

```
    syscall[SYSCALL_CURSOR] = k_cursor;
    syscall[SYSCALL_REFLUSH] = k_refresh;
```

登记光标移动和屏幕刷新。

L180

```
    syscall[SYSCALL_SLEEP] = k_sleep;
```

SLEEP号码映射到上面的k_sleep，形成用户睡眠请求到内核阻塞逻辑的桥。

L181–183

```
    syscall[SYSCALL_LOCK_INIT] = k_lock_init;
    syscall[SYSCALL_LOCK_ACQ] = k_lock_acq;
    syscall[SYSCALL_LOCK_RELEASE] = k_lock_rel;
```

登记锁初始化、获取和释放三个服务，复用Task2的内核实现。

L184–185

```
    syscall[SYSCALL_GET_TIMEBASE] = k_timebase;
    syscall[SYSCALL_GET_TICK] = k_ticks;
```

登记时钟频率和当前tick查询，用户无需直接访问内核变量。

L186

```
}
```

初始化结束。填表没有执行这些服务。

3. 系统调用表与handle_syscall：完整源码

P2:kernel/syscall/syscall.c • L1–L14 • 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+    1 #include <sys/syscall.h>
+    2 long (*syscall[NUM_SYSCALLS])();
+    3 void handle_syscall(regs_context_t *regs, uint64_t stval, uint64_t cause)
+    4 {
+    5     (void)stval; (void)cause;
+    6     uint64_t number = regs->regs[17]; /* a7 */
+    7     regs->sepc += 4; /* ecall is always a 32-bit instruction */
+    8     if (number >= NUM_SYSCALLS || !syscall[number]) {
+    9         regs->regs[10] = (reg_t)-38; /* ENOSYS */
+   10         return;
+   11     }
+   12     regs->regs[10] = syscall[number](regs->regs[10], regs->regs[11],
+   13                                      regs->regs[12], regs->regs[13], regs->regs[14]);
+   14 }
```

按原始行号逐句理解

L1

```
#include <sys/syscall.h>
```

引入号码和现场类型等声明。

L2

```
long (*syscall[NUM_SYSCALLS])();
```

定义函数指针数组syscall，供init_syscall填写。它不等于固定地址的BIOS函数入口。

L3–4

```
void handle_syscall(regs_context_t *regs, uint64_t stval, uint64_t cause)
{
```

定义同步系统调用处理函数，参数regs指向当前任务栈上的异常帧。

L5

```
    (void)stval; (void)cause;
```

显式标记本函数不使用的诊断参数，避免无用参数警告。

L6

```
    uint64_t number = regs->regs[17]; /* a7 */
```

regs[17]就是保存的a7，取出用户请求号码；不是从用户字符串里解析命令。

L7

```
    regs->sepc += 4; /* ecall is always a 32-bit instruction */
```

让保存的sepc前进4字节，跳过已经处理的ecall。ecall固定32位，若不前进，sret后会反复执行同一条请求。

L8

```
    if (number >= NUM_SYSCALLS || !syscall[number]) {
```

先检查号码边界，再检查该槽是否已注册。||短路避免越界读取syscall[number]。

L9–10

```
        regs->regs[10] = (reg_t)-38; /* ENOSYS */
        return;
```

不合法时把-38写进保存的a0槽并返回，表示ENOSYS；不能跳到空函数指针。

L11

```
    }
```

错误分支结束。

L12–13

```
    regs->regs[10] = syscall[number](regs->regs[10], regs->regs[11],
                                     regs->regs[12], regs->regs[13], regs->regs[14]);
```

以保存的a0到a4作为实参调用表项，得到long结果，再写回保存的a0。若服务会调度，这个C调用链会随当前任务栈一起暂停、以后恢复。

L14

```
}
```

处理完成后返回汇编入口，再由RESTORE_CONTEXT把新的a0带回用户。

把执行过程手算一遍

1. 用户a7=SYSCALL_SLEEP、a0=1 → exc_table[8] → handle_syscall → syscall[SYSCALL_SLEEP]=k_sleep → do_sleep(1)。
2. 用户请求未知号码：不调用任何服务，只把保存的a0置-38；恢复后invoke_syscall的return a0读到这个错误值。

应该看到什么，由哪里保证

- 睡眠/锁等用户接口能实际影响内核队列；探针对未注册或越界号码断言返回-38。

对应本次验证：stage3 / stage4（详细范围和结果见末尾）。

> **理解边界：**只给ecall的sepc加4；定时中断应该恢复被打断位置，不能无条件跳过一条用户指令。 call_bios的which、跳转表索引、系统调用号、scause是不同层次的信息。

Task 3 · 返回用户任务

15 恢复完整帧，最后用sret而不是ret

> **这一步要完成什么**
>
> 让系统调用的结果和被中断的计算状态都回到正确任务，并兼容首次启动。

与P1／前一步的关系：P1的任务crt0普通ret回加载器。P2进入U态后返回路径必须恢复CSR和通用寄存器，由sret按sepc/SPP决定PC与模式。

输入、输出与运行条件：入口时tp指当前PCB，sp指向其288字节异常帧；该帧可能刚由SAVE_CONTEXT产生，也可能是初始化时预制的。

1. RESTORE_CONTEXT：完整恢复宏

P2:arch/riscv/kernel/entry.S · L70–L109 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+   70 .macro RESTORE_CONTEXT
+   71     ld t0, OFFSET_REG_SSTATUS(sp)
+   72     csrw CSR_SSTATUS, t0
+   73     ld t0, OFFSET_REG_SEPC(sp)
+   74     csrw CSR_SEPC, t0
+   75     addi t0, sp, OFFSET_SIZE
+   76     sd t0, PCB_KERNEL_SP(tp)
+   77     csrw CSR_SSCRATCH, tp
+   78     ld ra, 8(sp)
+   79     ld gp, 24(sp)
+   80     ld t1, 48(sp)
+   81     ld t2, 56(sp)
+   82     ld s0, 64(sp)
+   83     ld s1, 72(sp)
+   84     ld a0, 80(sp)
+   85     ld a1, 88(sp)
+   86     ld a2, 96(sp)
+   87     ld a3, 104(sp)
+   88     ld a4, 112(sp)
+   89     ld a5, 120(sp)
+   90     ld a6, 128(sp)
+   91     ld a7, 136(sp)
+   92     ld s2, 144(sp)
+   93     ld s3, 152(sp)
+   94     ld s4, 160(sp)
+   95     ld s5, 168(sp)
+   96     ld s6, 176(sp)
+   97     ld s7, 184(sp)
+   98     ld s8, 192(sp)
+   99     ld s9, 200(sp)
+  100     ld s10, 208(sp)
+  101     ld s11, 216(sp)
+  102     ld t3, 224(sp)
+  103     ld t4, 232(sp)
```

```
+ 104     ld t5, 240(sp)
+ 105     ld t6, 248(sp)
+ 106     ld t0, OFFSET_REG_T0(sp)
+ 107     ld tp, OFFSET_REG_TP(sp)
+ 108     ld sp, OFFSET_REG_SP(sp)
+ 109 .endm
```

按原始行号逐句理解

L70

```
.macro RESTORE_CONTEXT
```

定义恢复宏，必须与保存帧的布局和初始帧构造一致。

L71–72

```
    ld t0, OFFSET_REG_SSTATUS(sp)
    csrw CSR_SSTATUS, t0
```

恢复保存的sstatus，包括SPP与SPIE等位；硬件陷入时SIE已关闭，正常处理路径不会在这里提前开放嵌套中断。

L73–74

```
    ld t0, OFFSET_REG_SEPC(sp)
    csrw CSR_SEPC, t0
```

恢复sepc。对于ecall，它已在处理器中加4；对于时钟中断，它保持被打断位置。

L75–76

```
    addi t0, sp, OFFSET_SIZE
    sd t0, PCB_KERNEL_SP(tp)
```

用当前帧起点加288得到撤销异常帧后的内核栈位置，并写回PCB.kernel_sp，供下次从U态陷入时切栈。

L77

```
    csrw CSR_SSCRATCH, tp
```

把sscratch重新保持为当前PCB，下一次异常不会依赖用户tp定位内核对象。

L78–79

```
    ld ra, 8(sp)
    ld gp, 24(sp)
```

从帧恢复任务ra和gp；恢复ra并不意味着这里要用ret离开，最终仍用sepc/sret。

L80–83

```
    ld t1, 48(sp)
    ld t2, 56(sp)
    ld s0, 64(sp)
    ld s1, 72(sp)
```

恢复t1、t2、s0、s1，与保存时的槽位逐项对应。

L84–91 恢复a0到a7。a0槽可能已被系统调用处理器改成返回值，其余参数寄存器保持原保存内容。

L92–101 恢复s2到s11，共10个64位寄存器；不能只凭测试打印正常就省略其中某个。

L102–105

```
    ld t3, 224(sp)
    ld t4, 232(sp)
    ld t5, 240(sp)
    ld t6, 248(sp)
```

恢复t3到t6，保证任意位置被抢占的用户临时计算可以继续。

L106

```
    ld t0, OFFSET_REG_T0(sp)
```

恢复原t0，此前用于计算和搬运CSR的临时值至此不再需要。

L107

```
    ld tp, OFFSET_REG_TP(sp)
```

最后阶段恢复原tp。用户tp即使不是PCB地址也会被保留，因此下一条不能再假设tp仍指内核PCB。

L108

```
    ld sp, OFFSET_REG_SP(sp)
```

恢复陷入前的sp。放在末尾是因为此前所有ld都还要从内核异常帧读取；过早换回用户sp会读错整份现场。

L109

```
.endm
```

宏结束；执行位置仍在内核，下一段sret才完成异常返回。

2. ret_from_exception：完整返回入口

P2:arch/riscv/kernel/entry.S · L167–L170 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+ 167 ENTRY(ret_from_exception)
+ 168     RESTORE_CONTEXT
+ 169     sret
+ 170 ENDPROC(ret_from_exception)
```

按原始行号逐句理解

L167

```
ENTRY(ret_from_exception)
```

统一恢复符号，首次启动的预制ra也指向它。

L168

```
    RESTORE_CONTEXT
```

把刚才完整宏展开到这里，恢复任务现场。

L169

```
    sret
```

sret采用sepc作为目标PC，并按保存的SPP选择回U态或S态。与普通ret读取ra的规则不同。

L170

```
ENDPROC(ret_from_exception)
```

返回入口符号结束。

把执行过程手算一遍

1. 正常ecall：保存现场 → 分发修改a0/sepc → 恢复 → sret → 用户ecall之后的指令。
2. 首次启动：switch_to取出预制ra → 本入口恢复预制帧 → sret → crt0。两条路径因为采用同一帧格式而汇合。

应该看到什么，由哪里保证

- Task3/4用户任务系统调用后继续循环；Task4寄存器检查与两个PASS同时检验保存、调度、恢复链。

对应本次验证：stage3 / stage4（详细范围和结果见末尾）。

> **理解边界：**恢复sp过早会让后面读错栈；恢复tp后还通过tp写PCB会把用户值当指针。 sret只恢复架构规定的部分状态；通用寄存器恢复工作仍必须由前面的ld完成。

Task 3 • 睡眠与唤醒

16 睡眠是进入等待队列，不是占着CPU空转

> **这一步要完成什么**
>
> 把用户请求的秒数转换成到期tick，让任务阻塞，到期后重新进入就绪队列。

与P1／前一步的关系：P1没有睡眠队列。P2复用Task2的阻塞/唤醒基础操作；到期检查由调度器调用，Task4再用时钟保证持续触发调度。

输入、输出与运行条件：time_base是每秒tick数，get_ticks是绝对计数；wakeup_time采用同一种tick单位。Task3本身没有时钟抢占。

1. 时间变量与查询函数：完整函数组

P2:kernel/sched/time.c • L5–L24 • 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+    5 uint64_t time_elapsed = 0;
+    6 uint64_t time_base = 0;
+    7
+    8 uint64_t get_ticks()
+    9 {
+   10     __asm__ __volatile__(
+   11         "rdtime %0"
+   12         : "=r"(time_elapsed));
+   13     return time_elapsed;
+   14 }
+   15
+   16 uint64_t get_timer()
+   17 {
+   18     return get_ticks() / time_base;
+   19 }
+   20
+   21 uint64_t get_time_base()
+   22 {
+   23     return time_base;
+   24 }
```

按原始行号逐句理解

L5–6

```
uint64_t time_elapsed = 0;
uint64_t time_base = 0;
```

定义当前读数存储和时钟频率。time_base由启动时读取设备树得到，不能一直保持0再拿它做除法。

L8–9

```
uint64_t get_ticks()
{
```

get_ticks返回硬件时间计数，不是已换算的秒数。

L10–12

```
    __asm__ __volatile__(
        "rdtime %0"
        : "=r"(time_elapsed));
```

rdtime读取计数，输出约束把结果写到time_elapsed；内联汇编volatile要求实际执行读取。

L13–14

```
    return time_elapsed;
}
```

返回读到的tick值。

L16–19

```
uint64_t get_timer()
{
    return get_ticks() / time_base;
}
```

get_timer把ticks除以每秒频率得到整数秒，较短间隔会被取整，所以睡眠到期比较直接用ticks更合适。

L21–24

```
uint64_t get_time_base()
{
    return time_base;
}
```

get_time_base返回频率，用户可经系统调用查询它，避免硬编码平台频率。

2. sys_sleep：完整用户接口

P2:tiny_libc/syscall.c · L82–L85 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+  82 void sys_sleep(uint32_t seconds)
+  83 {
+  84     invoke_syscall(SYSCALL_SLEEP, seconds, 0, 0, 0, 0);
+  85 }
```

按原始行号逐句理解

L82–83

```
void sys_sleep(uint32_t seconds)
{
```

接收uint32_t秒数，此接口用于本包的Task3/4用户态路径。

L84

```
    invoke_syscall(SYSCALL_SLEEP, seconds, 0, 0, 0, 0);
```

把秒数放到第一个参数，携带SLEEP号码进入系统调用封装。

L85

```
}
```

用户将来恢复后才从这里返回；接口没有在用户态忙等一段时间。

3. do_sleep：完整阻塞逻辑

P2:kernel/sched/sched.c · L39–L49 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+   39 void do_sleep(uint32_t seconds)
+   40 {
+   41     uint64_t flags = irq_save();
+   42     if (!seconds) {
+   43         do_scheduler();
+   44     } else {
+   45         current_running->wakeup_time = get_ticks() + (uint64_t)seconds * time_base;
+   46         do_block(&current_running->list, &sleep_queue);
+   47     }
+   48     irq_restore(flags);
+   49 }
```

按原始行号逐句理解

L39–40

```
void do_sleep(uint32_t seconds)
{
```

内核睡眠服务接收秒数，使用当前任务的PCB记录状态。

L41

```
    uint64_t flags = irq_save();
```

关本地中断保护截止时刻与入队过程。

L42–43

```
    if (!seconds) {
        do_scheduler();
```

秒数为0时直接调度，相当于让出一次，不创建一个未来等待条件。

L44

```
    } else {
```

正秒数进入真正的睡眠分支。

L45

```
        current_running->wakeup_time = get_ticks() + (uint64_t)seconds * time_base;
```

deadline=当前ticks+秒数×time_base。先转64位再乘，避免按32位乘法过早溢出；该字段存绝对tick值。

L46

```
        do_block(&current_running->list, &sleep_queue);
```

复用do_block，把当前任务标BLOCKED并移入sleep_queue，然后切到其他任务。

L47–49

```
    }
    irq_restore(flags);
}
```

被唤醒并重新调度回来后，恢复本层原中断状态再返回。

4. check_sleeping：完整到期检查

P2:kernel/sched/time.c · L34–L44 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+   34 void check_sleeping(void)
+   35 {
+   36     uint64_t now = get_ticks();
+   37     list_node_t *node = sleep_queue.next;
+   38     while (node != &sleep_queue) {
+   39         list_node_t *next = node->next;
+   40         pcb_t *task = list_entry(node, pcb_t, list);
+   41         if ((int64_t)(now - task->wakeup_time) >= 0) do_unblock(node);
+   42         node = next;
+   43     }
+   44 }
```

按原始行号逐句理解

L34–35

```
void check_sleeping(void)
{
```

调度器每次选择任务之前调用这个检查。

L36

```
    uint64_t now = get_ticks();
```

取一次当前tick作为本轮判定基准。

L37

```
    list_node_t *node = sleep_queue.next;
```

从睡眠队列的第一个节点开始遍历。

L38

```
    while (node != &sleep_queue) {
```

遇到哨兵头说明遍历结束，不把队列头当PCB。

L39

```
        list_node_t *next = node->next;
```

先保存next，因为do_unblock会删掉当前节点并改写它的连接关系。

L40

```
        pcb_t *task = list_entry(node, pcb_t, list);
```

从list成员地址取得当前睡眠任务PCB。

L41

```
        if ((int64_t)(now - task->wakeup_time) >= 0) do_unblock(node);
```

差值按有符号解释，非负表示截止时间已到，于是唤醒回READY队列；适用于正常、相差未跨半个64位计数范围的截止时刻。

L42

```
        node = next;
```

沿预先保存的next继续，不能在删链后再读node->next。

L43–44

```
    }
}
```

完成整个队列检查；一个到期任务被唤醒并不保证立刻获得CPU。

把执行过程手算一遍

1. 若time_base=10000000，当前ticks=25000000，请求sleep(2)，deadline就是45000000；到期前该任务不在ready_queue。
2. Task3：sleep阻塞后，coop_timer不断yield或idle轮询，给check_sleeping提供执行机会。若其他任务永不让出，Task3不能靠定时抢占强行唤醒执行；Task4补上这部分。

应该看到什么，由哪里保证

- sleep任务的计数暂停后继续，其他任务在它等待时仍能前进；探针还检查等待时长没有短于请求下界。

对应本次验证：stage3 / stage4 / idle（详细范围和结果见末尾）。

> **理解边界：**while(rdtime<deadline)若放在用户任务中只是在忙等，不等于内核阻塞睡眠。 到期、READY、真正RUNNING是三个不同阶段；不要把到期时间当成严格无误差的执行时刻。

Task 4 · 时钟抢占

17 任务不主动yield，内核也能收回CPU

> **这一步要完成什么**
>
> 安装时钟处理者，启用单个中断源，在每次定时中断中重设下一次时间并调度。

与P1／前一步的关系：Task1依赖主动yield，Task3依赖系统调用阻塞或主动调度。Task4用异步时钟中断触发同一调度器。

输入、输出与运行条件：tick单位、异常入口、PCB/队列与完整现场必须先正确。TIMER_INTERVAL是硬件tick数量，不是毫秒，也不是固定保证的每任务运行时长。

1. init_exception：完整注册流程

P2:kernel/irq/irq.c · L35–L46 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+   35 void init_exception()
+   36 {
+   37     for (int i = 0; i < EXCC_COUNT; ++i) exc_table[i] = handle_other;
+   38     for (int i = 0; i < IRQC_COUNT; ++i) irq_table[i] = handle_other;
+   39 #if LAB_STAGE >= 3
+   40     exc_table[EXCC_SYSCALL] = handle_syscall;
+   41 #endif
+   42 #if LAB_STAGE >= 4
+   43     irq_table[IRQC_S_TIMER] = handle_irq_timer;
+   44 #endif
+   45     setup_exception();
+   46 }
```

按原始行号逐句理解

L35–36

```
void init_exception()
{
```

定义异常/中断表初始化入口，由main在允许任务运行前调用。

L37–38

```
    for (int i = 0; i < EXCC_COUNT; ++i) exc_table[i] = handle_other;
    for (int i = 0; i < IRQC_COUNT; ++i) irq_table[i] = handle_other;
```

先把全部槽位设为handle_other，使没有专门处理的原因进入明确诊断，而不是随机跳转。

L39–41

```
#if LAB_STAGE >= 3
    exc_table[EXCC_SYSCALL] = handle_syscall;
#endif
```

阶段3及以后才把U态ecall异常号登记到handle_syscall；阶段1/2不靠该路径提供普通服务。

L42–44

```
#if LAB_STAGE >= 4
    irq_table[IRQC_S_TIMER] = handle_irq_timer;
#endif
```

阶段4才登记S态定时中断处理器。仅存在函数定义不够，必须在irq_table里接到对应原因号。

L45–46

```
    setup_exception();
}
```

调用汇编setup_exception安装stvec和sscratch等入口约定，完成注册；还没有设置第一轮定时器或全局开启中断。

2. 中断源与全局开关：完整汇编函数组

P2:arch/riscv/kernel/entry.S · L111–L128 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+ 111 ENTRY(enable_preempt)
+ 112     li t0, SIE_STIE
+ 113     csrs CSR_SIE, t0
+ 114     ret
+ 115 ENDPROC(enable_preempt)
+ 116 ENTRY(disable_preempt)
+ 117     li t0, SIE_STIE
+ 118     csrc CSR_SIE, t0
+ 119     ret
+ 120 ENDPROC(disable_preempt)
+ 121 ENTRY(enable_interrupt)
+ 122     csrsi CSR_SSTATUS, 2
+ 123     ret
+ 124 ENDPROC(enable_interrupt)
+ 125 ENTRY(disable_interrupt)
+ 126     csrci CSR_SSTATUS, 2
+ 127     ret
+ 128 ENDPROC(disable_interrupt)
```

按原始行号逐句理解

L111–112

```
ENTRY(enable_preempt)
    li t0, SIE_STIE
```

进入enable_preempt，准备SIE_STIE掩码，即S态timer中断源使能位。

L113–115

```
    csrs CSR_SIE, t0
    ret
ENDPROC(enable_preempt)
```

将STIE置位并返回，只开放这个中断源；不能据此说已经无条件打开全局SIE或设置了定时器。

L116–120

disable_preempt清同一个STIE位，关闭该来源，其他来源位保持原值。

L121–124

```
ENTRY(enable_interrupt)
    csrsi CSR_SSTATUS, 2
    ret
ENDPROC(enable_interrupt)
```

enable_interrupt把sstatus的SIE位设为1，控制S态全局响应；与sie中的STIE是两处不同的位。

L125–128

```
ENTRY(disable_interrupt)
    csrci CSR_SSTATUS, 2
    ret
ENDPROC(disable_interrupt)
```

disable_interrupt清全局SIE，用于idle选择和等待的原子流程；它不删除已登记的处理者，也不取消未来的定时器。

3. 本实现的定时间隔常量

P2:include/os/time.h · L33–L33 · 完整常量定义
相对P1新增的文件；课程框架保留的关键依赖

```
+   33 #define TIMER_INTERVAL 150000
```

按原始行号逐句理解

L33

```
#define TIMER_INTERVAL 150000
```

TIMER_INTERVAL=150000，单位为rdtime ticks。若time_base是10MHz，约为15ms；平台频率不同就不同，调度开销和阻塞也会影响实际CPU时间。

4. handle_irq_timer：完整时钟回调

P2:kernel/irq/irq.c · L27–L33 · 完整函数／连续实现块
相对P1新增的文件；课程框架基础上修改／补全

```
+   27 void handle_irq_timer(regs_context_t *regs, uint64_t stval, uint64_t scause)
+   28 {
+   29     (void)regs; (void)stval; (void)scause;
```

```
+   30     bios_set_timer(get_ticks() + TIMER_INTERVAL);
+   31     screen_reflush();
+   32     do_scheduler();
+   33 }
```

按原始行号逐句理解

L27–28

```
void handle_irq_timer(regs_context_t *regs, uint64_t stval, uint64_t scause)
{
```

由统一中断分发表调用，原任务完整现场已由SAVE_CONTEXT保存。

L29

```
    (void)regs; (void)stval; (void)scause;
```

标记本处理器不使用的参数，避免编译警告。

L30

```
    bios_set_timer(get_ticks() + TIMER_INTERVAL);
```

读取当前ticks，加TIMER_INTERVAL，重新请求下一次定时器。若只设置启动时第一次中断，就无法持续抢占。

L31

```
    screen_reflush();
```

刷新屏幕缓存，让任务输出显示出来；它不是调度的本质条件。

L32

```
    do_scheduler();
```

进入同一个do_scheduler。仍RUNNING的普通任务回队尾，选队首，必要时switch_to；这条路径无需用户主动sys_yield。

L33

```
}
```

该任务以后被恢复时，中断处理函数才继续返回，最终经汇编sret回到被打断的PC。

把执行过程手算一遍

1. 时钟到期 → scause表明S timer → 保存完整现场 → irq_table[5] → 重设下一次时间 → do_scheduler → 切换。
2. 注意“有两个计数器在打印”只是弱证据：打印本身也会进内核。probe1/2额外安排无系统调用的CPU计算，要求双方在对方计算结束前都开始，才更直接覆盖异步抢占。

应该看到什么，由哪里保证

- 原print1/print2的sys_yield已注释，两个计数仍持续前进；更强探针先后出现两个start，再完成各自cpu-loop-ok与PASS。

对应本次验证：stage4 / course（详细范围和结果见末尾）。

> **理解边界：**只写handle_irq_timer但没有注册、STIE或首次定时请求，函数不会自动运行。 不能给中断的sepc统一加4，否则会跳过尚未执行完成的用户指令；只有ecall同步请求跳过自身。

Task 1—4 • 启动顺序与idle

18 最后接总线：准备完成后才让任务运行

> **这一步要完成什么**
>
> 用main把所有机制接成实际执行链，并在用户任务全部阻塞时保留一个能唤醒它们的idle任务。

与P1／前一步的关系：P1 main围绕键盘任务名循环。P2 main先初始化多个任务和内核服务，再由pid0进入调度循环；不再把普通任务逐个call到结束。

输入、输出与运行条件：输入是bootblock传来的任务数和镜像任务表偏移；执行时处于S态、使用与pid0一致的启动栈，初始中断关闭。

1. P2 main：完整函数，包含idle逻辑

P2:init/main.c • L188–L219 • 完整函数／连续实现块
相对P1修改的文件；课程框架基础上修改／补全

```
+ 188 int main(int count, uint32_t app_offset)
+ 189 {
  190     init_jmptab();
+ 191     init_task_info(count, app_offset);
+ 192     init_pcb();
+ 193     time_base = bios_read_fdt(TIMEBASE);
+ 194     assert(time_base > 0);
+ 195     init_locks();
+ 196     init_syscall();
+ 197     init_exception();
+ 198     init_screen();
+ 199     printk("[INIT] A-Core stage %d, %d tasks, timebase=%lu\n\r",
+ 200            LAB_STAGE, task_num, time_base);
+ 201 #if LAB_STAGE >= 4
+ 202     bios_set_timer(get_ticks() + TIMER_INTERVAL);
+ 203     enable_preempt();
+ 204 #endif
+ 205     /************************************************************/
+ 206     /* Do not touch this comment. Reserved for future projects. */
+ 207     /************************************************************/
+ 208     while (1) {
+ 209         disable_interrupt();
+ 210         do_scheduler();
+ 211 #if LAB_STAGE >= 4
+ 212         /* WFI may wake on a pending individually enabled interrupt even
+ 213          * with global SIE clear. Enable after WFI to service it safely. */
+ 214         __asm__ volatile ("wfi");
+ 215         enable_interrupt();
+ 216 #endif
+ 217         /* Stage 3 deliberately polls here when every task is asleep. */
+ 218     }
+ 219 }
```

按原始行号逐句理解

L188–189

```
int main(int count, uint32_t app_offset)
{
```

接收引导传参。count不是当前pid，app_offset是磁盘镜像字节偏移而不是内存指针。

L190

```
    init_jmptab();
```

先填底层跳转表，使后面的BIOS封装能调用已经准备好的服务。

L191

```
    init_task_info(count, app_offset);
```

把镜像任务记录读入tasks数组，并检查数量、名称结尾、入口和大小。

L192

```
    init_pcb();
```

加载各任务，分配独立栈与PCB，构造初始帧并排入就绪队列。当前仍由pid0执行，不能在半初始化中途抢占。

L193–194

```
    time_base = bios_read_fdt(TIMEBASE);
    assert(time_base > 0);
```

从设备树取得time_base并检查非零，后面的秒数转换与用户计时都依赖它。

L195

```
    init_locks();
```

初始化锁状态、owner、used和各等待队列，避免任务第一次竞争时读未准备的数据。

L196

```
    init_syscall();
```

建立内核系统调用表。

L197

```
    init_exception();
```

建立异常/中断分发表并安装stvec等入口；这一步与仅填写函数指针表不同。

L198

```
    init_screen();
```

初始化屏幕缓存和光标状态。

L199–200

```
    printk("[INIT] A-Core stage %d, %d tasks, timebase=%lu\n\r",
           LAB_STAGE, task_num, time_base);
```

打印阶段、任务数、频率。它证明初始化走到了这里，仍不能独自证明切换和抢占正确。

L201–204

```
#if LAB_STAGE >= 4
    bios_set_timer(get_ticks() + TIMER_INTERVAL);
    enable_preempt();
#endif
```

阶段4先设置第一轮定时器，再启用STIE中断源。阶段1—3编译时不包含这两项定时抢占启动。

L205–207

```
    /************************************************************/
    /* Do not touch this comment. Reserved for future projects. */
    /************************************************************/
```

保留注释，不生成运行步骤。

L208

```
    while (1) {
```

pid0持续保留在内核中，不会main返回后丢掉所有任务都阻塞时的退路。

L209

```
        disable_interrupt();
```

每次idle选择前关闭全局SIE，保护检查就绪队列到等待之间的流程。

L210

```
        do_scheduler();
```

调度普通任务；若没有可运行对象就选回pid0，并在每次选择前检查sleep队列。

L211

```
#if LAB_STAGE >= 4
```

阶段4采用等待中断的idle分支。

L212–214

```
        /* WFI may wake on a pending individually enabled interrupt even
         * with global SIE clear. Enable after WFI to service it safely. */
        __asm__ volatile ("wfi");
```

即使全局SIE仍关闭，架构允许WFI因单独已使能且pending的中断而恢复。先关中断检查再WFI，避免“刚检查无人可跑就错过一次唤醒”的窗口。

L215

```
        enable_interrupt();
```

WFI恢复后才开全局SIE，让已经到达的定时中断进入处理器，再由它重新设置定时器和调度。

L216

```
#endif
```

结束阶段4分支。

L217

```
        /* Stage 3 deliberately polls here when every task is asleep. */
```

阶段3没有timer IRQ，pid0就不断循环调度/检查睡眠截止时刻，而不是WFI等一个尚未启用的中断。

L218–219

```
    }
}
```

继续idle循环，普通任务可在以后重新被调度。若所有用户任务睡眠，系统仍有执行和唤醒路径。

把执行过程手算一遍

1. 完整首次启动链：bootblock → head._start清BSS/设栈 → main填表/建任务 → do_scheduler → switch_to → ret_from_exception → sret → 任务crt0 → main。
2. 全部任务睡眠：就绪队列空 → pid0 → Task3轮询或Task4 WFI/时钟 → check_sleeping → 到期任务READY → 再调度。

应该看到什么，由哪里保证

- 单probe实验中唯一用户任务sleep后仍能醒来并打印PASS；没有pid0或时钟未重设时，这条路径会停滞。

对应本次验证：idle / stage3 / stage4（详细范围和结果见末尾）。

> **理解边界：**不能一进入main就开启定时中断：中断可能访问尚未初始化的队列/表。 WFI在真实架构上允许虚假唤醒，因此外层必须循环重新检查，不能把每次WFI恢复都当成某个任务已到期。

构建与装载前提 · 聚焦关键变化

19 哪些任务入镜像、入口放哪里、装完才可取指

> **这一步要完成什么**
>
> 让本书讲的任务、映像地址、阶段开关与运行时PCB一一对应，避免代码正确却根本没有构建进镜像。

与P1／前一步的关系：P1会按名字加载一个任务；P2构建时用TASKS选定启动任务集合，保持每个入口与镜像元数据一致。riscv.lds和task_info_t基本协议沿用，不因文件存在差异就重新解释全部内容。

输入、输出与运行条件：同一轮make run要提供相同LAB_STAGE/TASKS。入口从0x52000000开始，每个任务一个64KiB映像槽；磁盘内容仍紧凑存放，不能把槽位大小当磁盘扇区数。

1. 阶段与任务选择：完整条件块

P2:Makefile · L14–L38 · 完整构建条件块
相对P1修改的文件；课程框架基础上修改／补全

```
+  14 LAB_STAGE ?= 4
+  15 ifeq ($(filter $(LAB_STAGE),1 2 3 4),)
+  16 $(error LAB_STAGE must be 1, 2, 3 or 4)
+  17 endif
+  18 ifeq ($(LAB_STAGE),1)
+  19 TASKS ?= coop1 coop2
+  20 else ifeq ($(LAB_STAGE),2)
+  21 TASKS ?= mutex1 mutex2
+  22 else ifeq ($(LAB_STAGE),3)
+  23 TASKS ?= sleep coop_timer
+  24 else
+  25 TASKS ?= print1 print2 lock1 lock2 sleep timer
+  26 endif
+  27 COURSE_TASKS = fly lock1 lock2 print1 print2 sleep timer
+  28 VALIDATION_TASKS = coop1 coop2 mutex1 mutex2 coop_timer probe1 probe2
+  29 VALID_TASKS = $(COURSE_TASKS) $(VALIDATION_TASKS)
+  30 ifneq ($(filter-out $(VALID_TASKS),$(TASKS)),)
+  31 $(error Unknown A-Core task: $(filter-out $(VALID_TASKS),$(TASKS)))
+  32 endif
+  33 ifeq ($(strip $(TASKS)),)
+  34 $(error TASKS must not be empty)
+  35 endif
+  36 ifneq ($(words $(TASKS)),$(words $(sort $(TASKS))))
+  37 $(error TASKS must not contain duplicates)
+  38 endif
```

按原始行号逐句理解

L14

```
LAB_STAGE ?= 4
```

默认构建阶段4；命令行LAB_STAGE=1等可覆盖默认值。

L15–17

```
ifeq ($(filter $(LAB_STAGE),1 2 3 4),)
$(error LAB_STAGE must be 1, 2, 3 or 4)
endif
```

只允许1到4，输入其他值明确报错，防止编出半套不明配置。

L18–19

```
ifeq ($(LAB_STAGE),1)
TASKS ?= coop1 coop2
```

阶段1默认选择两个主动yield的协作探针。

L20–21

```
else ifeq ($(LAB_STAGE),2)
TASKS ?= mutex1 mutex2
```

阶段2选择两个互斥锁竞争探针。

L22–23

```
else ifeq ($(LAB_STAGE),3)
TASKS ?= sleep coop_timer
```

阶段3选择sleep和持续yield计时任务，适合在无时钟抢占下观察阻塞与恢复。

L24–26

```
else
TASKS ?= print1 print2 lock1 lock2 sleep timer
endif
```

阶段4默认六个课程任务print1/2、lock1/2、sleep、timer。

L27

```
COURSE_TASKS = fly lock1 lock2 print1 print2 sleep timer
```

声明本实现支持的课程任务集合；Task5的fly1到fly5不在这里。

L28–29

```
VALIDATION_TASKS = coop1 coop2 mutex1 mutex2 coop_timer probe1 probe2
VALID_TASKS = $(COURSE_TASKS) $(VALIDATION_TASKS)
```

补上验证任务集合，合并成合法名称白名单。

L30–32

```
ifneq ($(filter-out $(VALID_TASKS),$(TASKS)),)
$(error Unknown A-Core task: $(filter-out $(VALID_TASKS),$(TASKS)))
endif
```

TASKS中出现未知名字即报错，不能静默漏掉一个用户以为已加入的任务。

L33–35

```
ifeq ($(strip $(TASKS)),)
$(error TASKS must not be empty)
endif
```

任务列表为空时拒绝构建。

L36–38

```
ifneq ($(words $(TASKS)),$(words $(sort $(TASKS))))
$(error TASKS must not contain duplicates)
endif
```

比较原列表与去重列表长度，拒绝重复任务名，使名称与入口槽对应关系唯一。

2. 裸机编译／链接约定：完整标志定义

P2:Makefile · L40–L45 · 完整构建变量块
相对P1修改的文件；课程框架基础上修改／补全

```
+  40 RISCV_ARCH ?= rv64imac_zicsr_zifencei
+  41 CFLAGS = -O1 -g -Wall -Wextra -Wno-unused-parameter -ffreestanding -fno-builtin \
+  42          -nostdlib -nostdinc -fno-pie -fno-stack-protector -fno-common \
+  43          -mcmodel=medany -msmall-data-limit=0 -mno-relax \
+  44          -march=$(RISCV_ARCH) -mabi=lp64 -DLAB_STAGE=$(LAB_STAGE)
+  45 LDFLAGS = -static -no-pie -Wl,--build-id=none -Wl,--no-relax -T riscv.lds
```

按原始行号逐句理解

L40

```
RISCV_ARCH ?= rv64imac_zicsr_zifencei
```

明确RV64IMAC以及CSR/fence.i扩展，不使用浮点ABI，和现场保存范围一致。

L41–42

```
CFLAGS = -O1 -g -Wall -Wextra -Wno-unused-parameter -ffreestanding -fno-builtin \
         -nostdlib -nostdinc -fno-pie -fno-stack-protector -fno-common \
```

采用独立内核编译方式，无标准启动/库和标准头搜索，并关闭PIE、栈保护与common合并等默认机制；本项目自带启动和运行库。

L43–44

```
         -mcmodel=medany -msmall-data-limit=0 -mno-relax \
         -march=$(RISCV_ARCH) -mabi=lp64 -DLAB_STAGE=$(LAB_STAGE)
```

medany配合地址布局；关闭小数据gp寻址和松弛，指定lp64及编译阶段宏。不能假定未初始化gp时仍能运行依赖gp的产物。

L45

```
LDFLAGS = -static -no-pie -Wl,--build-id=none -Wl,--no-relax -T riscv.lds
```

链接采用静态非PIE，禁止build-id与松弛，使用riscv.lds。P1在当前默认工具链下曾出现PT_LOAD起点比入口低，不能只凭make返回0判断映像可直接运行。

3. 链接入口与打包：完整任务规则及镜像规则

P2:Makefile · L77–L83 · 完整构建规则块
相对P1修改的文件；课程框架基础上修改／补全

```
+  77 build/%: %.c build/crt0.o build/libtinyc.a riscv.lds $(HEADERS) build/config
+  78     $(CC) $(CFLAGS) $(UINC) $(LDFLAGS) -Wl,--defsym=TEXT_START=$$(python3 -c 'import sy
+   > s; print(hex(0x52000000+sys.argv[2:].index(sys.argv[1])*0x10000))' '$*' $(TASKS)) -o $@
+   >  build/crt0.o $< -Lbuild -ltinyc
+  79 build/createimage: tools/createimage.c
+  80     $(HOST_CC) -O2 -g -Wall -Wextra -Werror $< -o $@
+  81 elf: build/bootblock build/main $(USER_ELF)
+  82 image: elf build/createimage
+  83     cd build && ./createimage --extended bootblock main $(TASKS)
```

按原始行号逐句理解

L77

```
build/%: %.c build/crt0.o build/libtinyc.a riscv.lds $(HEADERS) build/config
```

每个build/任务依赖对应源文件、crt0、自带库、链接脚本、头文件和构建配置。

L78

```
    $(CC) $(CFLAGS) $(UINC) $(LDFLAGS) -Wl,--defsym=TEXT_START=$$(python3 -c 'import sys; print
(hex(0x52000000+sys.argv[2:].index(sys.argv[1])*0x10000))' '$*' $(TASKS)) -o $@ build/crt0.o $<
-Lbuild -ltinyc
```

按任务名在TASKS中的位置计算0x52000000+index×0x10000，作为TEXT_START。每个任务独立链接；不是等程序运行时才临时改入口。

L79–80

```
build/createimage: tools/createimage.c
    $(HOST_CC) -O2 -g -Wall -Wextra -Werror $< -o $@
```

从宿主C源码编译createimage，运行在Linux宿主机上，不是运行在RISC-V任务里。

L81

```
elf: build/bootblock build/main $(USER_ELF)
```

elf目标要求bootblock、内核与全部所选任务都构建完成。

L82–83

```
image: elf build/createimage
    cd build && ./createimage --extended bootblock main $(TASKS)
```

镜像目标依赖ELF与工具，再进入build目录按bootblock、main、TASKS顺序打包，任务记录顺序与链接槽位顺序一致。

4. init_task_info：完整任务记录检查

P2:init/main.c · L112–L122 · 完整函数／连续实现块
相对P1修改的文件；课程框架基础上修改／补全

```
+ 112 static void init_task_info(int count, uint32_t offset)
+ 113 {
+ 114     assert(count > 0 && count <= TASK_MAXNUM && count <= NUM_MAX_TASK);
+ 115     task_num = count;
+ 116     load_img_bytes(offset, count * sizeof(task_info_t), tasks);
```

```
+ 117     for (int i = 0; i < count; ++i) {
+ 118         assert(tasks[i].task_name[TASK_NAME_LEN-1] == 0);
+ 119         assert(tasks[i].entrypoint == TASK_MEM_BASE + (ptr_t)i * TASK_SIZE);
+ 120         assert(tasks[i].size > 0 && tasks[i].size <= TASK_SIZE);
+ 121     }
+ 122 }
```

按原始行号逐句理解

L112–113

```
static void init_task_info(int count, uint32_t offset)
{
```

接收引导传入的任务数和任务表字节偏移。

L114

```
    assert(count > 0 && count <= TASK_MAXNUM && count <= NUM_MAX_TASK);
```

限制任务数为正并不超过磁盘记录/PCB数组容量，避免P1那类未检查数量就写数组的风险。

L115

```
    task_num = count;
```

保存全局task_num，后续创建、查找与指针检查共用它。

L116

```
    load_img_bytes(offset, count * sizeof(task_info_t), tasks);
```

按字节读取count条固定布局任务记录；这个tasks数组不是PCB数组。

L117

```
    for (int i = 0; i < count; ++i) {
```

逐项检查每条记录的运行前提。

L118

```
        assert(tasks[i].task_name[TASK_NAME_LEN-1] == 0);
```

确认32字节名字槽最后一个字节为0，后续strcmp才有已约束的字符串终点。

L119

```
        assert(tasks[i].entrypoint == TASK_MEM_BASE + (ptr_t)i * TASK_SIZE);
```

确认入口等于基址加索引槽位，与Makefile规则一致。

L120

```
        assert(tasks[i].size > 0 && tasks[i].size <= TASK_SIZE);
```

文件跨度应大于0且不超过64KiB任务槽。

L121–122

```
    }
}
```

完成校验。这里检查的是若干必要条件，不是任意损坏磁盘的完整安全验证。

5. load_task_img：完整P2按名加载

P2:kernel/loader/loader.c • L26–L38 • 完整函数／连续实现块
相对P1修改的文件；课程框架基础上修改／补全

```
+   26 uint64_t load_task_img(char *name)
    27 {
+   28     for (int i = 0; i < task_num; ++i) {
+   29         if (!strcmp(name, tasks[i].task_name)) {
+   30             assert(tasks[i].size <= TASK_SIZE);
+   31             load_img_bytes(tasks[i].offset, tasks[i].size,
+   32                            (void *)tasks[i].entrypoint);
+   33             __asm__ volatile ("fence.i" ::: "memory");
    34             return tasks[i].entrypoint;
    35         }
+   36     }
+   37     return 0;
    38 }
```

按原始行号逐句理解

L26–27

```
uint64_t load_task_img(char *name)
{
```

接收任务名，返回入口；接口基本延续P1。

L28

```
    for (int i = 0; i < task_num; ++i) {
```

在已经检查并读入的任务记录中逐条查找。

L29

```
        if (!strcmp(name, tasks[i].task_name)) {
```

strcmp返回0表示名称完全相等；!把它转成条件真。

L30

```
            assert(tasks[i].size <= TASK_SIZE);
```

再次约束映像大小不超过任务槽。

L31–32

```
            load_img_bytes(tasks[i].offset, tasks[i].size,
                           (void *)tasks[i].entrypoint);
```

复用跨扇区字节读取函数，把镜像offset/size对应数据搬到entrypoint内存地址。

L33

```
            __asm__ volatile ("fence.i" ::: "memory");
```

新增关键fence.i，使后续取指能看到刚由数据写入路径装载的指令字节；它不是C函数调用或普通字符串操作。

L34

```
            return tasks[i].entrypoint;
```

读完并同步后才把入口交给初始化PCB的调用者；这里仍没有执行任务。

L35–38

```
        }
    }
    return 0;
}
```

未命中返回0，外层会断言，不能把失败值当跳转目标。

把执行过程手算一遍

1. 本包还重写createimage以保留PT_LOAD相对虚拟地址间隙、检查大小/入口/数量、补齐尾扇区。这里把它列为装载正确性的前提，重点仍是Task1—4的运行机制。
2. 同样的任务名字只有在“编译进入TASKS → 正确链接 → 镜像记录 → init_task_info → load_task_img → PCB”整条链都接上后，才会真正运行。

应该看到什么，由哪里保证

- 启动LOAD列表与TASKS一致；镜像10项测试覆盖正常布局及超限/错误输入。

对应本次验证：image / course（详细范围和结果见末尾）。

> **理解边界：**修改TASKS之后运行时忘记同样参数，会得到不同任务组合。 编译成功不能代替入口、段布局和运行检查；当前不把通用QEMU测试当课程BIOS/块设备验收。

服务边界 • 输出与错误返回

20 字符串输出为什么还要检查指针

> **这一步要完成什么**
>
> 在系统调用分发之后，确保内核读取字符串时限定在当前任务允许的映像槽或栈范围内。

与P1／前一步的关系：P1主要信任同S态代码直接交给BIOS的地址。Task3/4从U态传来一个整数地址，内核不能只因为它是char*就盲目遍历。

输入、输出与运行条件：这是当前无页表隔离实现中的软件范围检查。它不是完整进程内存保护，也没有验证任意用户程序的所有访存。

1. k_write：完整用户字符串检查与输出

P2:init/main.c • L34–L57 • 完整函数／连续实现块
相对P1修改的文件；课程框架基础上修改／补全

```
+   34 static long k_write(long p, long b, long c, long d, long e)
+   35 {
+   36     /* No page tables in P2. Validate the complete string against this task's
+   37      * user image or user stack before dereferencing it, also for loadbootd. */
+   38     uint64_t begin = (uint64_t)p, limit = 0;
+   39     for (int i = 0; i < task_num; ++i) {
+   40         if (current_running->pid == i + 1 && begin >= tasks[i].entrypoint &&
+   41             begin < tasks[i].entrypoint + TASK_SIZE)
+   42             limit = tasks[i].entrypoint + TASK_SIZE;
+   43     }
+   44     if (begin >= current_running->user_stack_base &&
+   45         begin < current_running->user_stack_base + PAGE_SIZE)
+   46         limit = current_running->user_stack_base + PAGE_SIZE;
+   47     if (!limit) return -14;
+   48     char *text = (char *)p;
+   49     uint64_t i = 0;
+   50     while (begin + i < limit && text[i]) ++i;
+   51     if (begin + i == limit) return -14;
+   52     screen_write(text);
+   53 #if LAB_STAGE < 4
+   54     screen_reflush();
+   55 #endif
+   56     return 0;
+   57 }
```

按原始行号逐句理解

L34–35

```
static long k_write(long p, long b, long c, long d, long e)
{
```

统一系统服务签名，第一个参数p携带用户字符串地址，其余参数本服务不用。

L36–37

```
    /* No page tables in P2. Validate the complete string against this task's
     * user image or user stack before dereferencing it, also for loadbootd. */
```

注释明确没有页表；检查目标是完整字符串位于当前任务映像槽或用户栈内。

L38

```
    uint64_t begin = (uint64_t)p, limit = 0;
```

把地址转为64位整数，并以limit=0表示暂未找到允许范围。

L39

```
    for (int i = 0; i < task_num; ++i) {
```

遍历磁盘任务记录，找当前pid对应的映像槽。

L40–42

```
        if (current_running->pid == i + 1 && begin >= tasks[i].entrypoint &&
            begin < tasks[i].entrypoint + TASK_SIZE)
            limit = tasks[i].entrypoint + TASK_SIZE;
```

当前创建顺序保证pid=i+1；若地址位于该任务入口到入口+TASK_SIZE之间，取得槽的上界。允许范围是整个槽，不只是实际文件字节数。

L43

```
    }
```

映像范围检查结束。

L44–46

```
    if (begin >= current_running->user_stack_base &&
        begin < current_running->user_stack_base + PAGE_SIZE)
        limit = current_running->user_stack_base + PAGE_SIZE;
```

若地址位于当前任务的一页用户栈内，取用户栈的上界；常见局部格式化缓冲区也就能输出。

L47

```
    if (!limit) return -14;
```

两个范围都不匹配则返回-14(EFAULT)，在解引用之前拒绝。

L48–49

```
    char *text = (char *)p;
    uint64_t i = 0;
```

仅在范围已认可后把p解释为char*，准备逐字节找字符串结束符。

L50

```
    while (begin + i < limit && text[i]) ++i;
```

先检查地址尚未到limit，再读取text[i]。&&短路避免先越界读取；找到零字节或到界限就停。

L51

```
    if (begin + i == limit) return -14;
```

到边界仍未找到结束符，则拒绝输出并返回EFAULT，不能无界扫描别的任务内存。

L52

```
    screen_write(text);
```

完整字符串通过检查后才写入屏幕缓冲；此处不是直接把字符串地址交给用户来调用内核函数。

L53–55

```
#if LAB_STAGE < 4
    screen_reflush();
#endif
```

阶段1—3立即刷新。阶段4通常依赖周期时钟或显式刷新，因而一次写入未必立刻在屏幕形成新字符。

L56–57

```
    return 0;
}
```

返回0表示该服务请求完成；经过系统调用时由handle_syscall写进用户a0。

把执行过程手算一遍

1. 合法栈缓冲区 → 判断地址在user_stack_base起的一页内 → 找到结束符 → screen_write；不合法地址则在text[i]读取前返回-14。
2. probe用错误字符串指针和非法系统调用号分别检查EFAULT与ENOSYS，两种错误来自不同层。

应该看到什么，由哪里保证

- 合法输出能显示；错误字符串请求返回-14而非随便解引用，且其后任务还能继续。

对应本次验证：stage4（详细范围和结果见末尾）。

> **理解边界：**这不意味着所有用户访存都被保护；无页表隔离的边界必须明确。 若将来改变pid与任务槽对应关系，这段pid==i+1的前提也必须一起更新。

来源、选择范围与原始许可

本书是教学解释，不是新的内核实现。对比来源为projects/Project1/Project1-current.tar.gz与projects/Project2/Project2-Acore.tar.gz，另外用Project2-framework-to-implementation.diff区分框架与完成实现。原上传源与工程源码均未改动。

材料	SHA-256
Project1-current.tar.gz	11cffe9e85474e2f24b3b6ea528495b4b332f2c0839a064c336a92c8888d4f94
Project2-Acore.tar.gz	2ab7415ff127705c660a5ed35255e53beab81bf185b5ddcde4c9556a5cab9a20
Project1-to-Project2.diff	fff68828dc2d5a09b0965041e6f2475ff826aaf030989b1d56d50a7c70d4e3df
Project2-framework-to-implementation.diff	494d9a0e96f1e8d557b6d849cc8104e2cd647532ad053423df5e3dcf210b8a15

选择范围：包括实际完成任务所需的函数、宏、类型和构建逻辑；不承诺覆盖整个工程所有差异。844行是代码行次，重复展示的统一返回入口计入两次；coverage.json另记录去重行数。每段都从归档连续取出，不把拼接后的伪代码当原函数。

没有展开讲解的一般格式化库、屏幕绘制细节、注释排版和保留原样P1节布局，可回到源码包查阅。工具链与镜像检查在第19步交代关键前提，不替代完整镜像格式手册。

框架源码保留原作者版权与许可。下列为本书所引课程C/头文件中的原始许可声明；各原文件的其他许可标记仍以压缩包为准。

```
           Copyright (C) 2018 Institute of Computing Technology, CAS
              Author : Han Shukai (email : hanshukai@ict.ac.cn)
* * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * *  * * * * * * * * * * * *
       Process scheduling related content, such as: scheduler, process blocking,
                process wakeup, process creation, process kill, etc.
* * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * *  * * * * * * * * * * * *
Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in
all copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
THE SOFTWARE.
```

最后：现象、原因、验证一一对应

观察什么	由什么代码保证	只看到什么还不够
两个协作计数都增长	步骤03队列、06调度、07切栈、08主动yield共同形成闭环	仅有两个LOAD打印不证明两任务已执行
锁等待者不能提前进临界区	步骤11先BLOCKED再等待，释放时先交owner再READY，底层锁保持占用	只跑无竞争的单任务锁测试
系统调用后回到下一条用户指令	步骤12传参、13保存、14写a0并sepc+=4、15恢复和sret	只确认ecall进入内核但没有正确返回
睡眠时其他任务继续，到期后恢复	步骤16设置绝对tick并阻塞，调度前检查到期并回READY	用户忙等若干秒后输出
没有主动yield也能切换	步骤17周期重设timer、完整现场保护、调用do_scheduler	仅看printf计数，不覆盖无系统调用的计算段
全体用户任务sleep后仍会醒	步骤18空队列选pid0，Task3轮询或Task4 WFI/IRQ重新调度	只有两任务互相轮转、从未遇到空队列
非法调用号与坏字符串得到明确错误	步骤14未注册/越界号码返回-38；步骤20越界字符串返回-14	没有异常不等于指针检查确实执行

本次重新运行的证据

检查	本次结果	退出码
镜像布局与错误输入	10项通过	0
Task 1 通用QEMU适配运行	按运行器实际输出断言通过	0
Task 2 通用QEMU适配运行	按运行器实际输出断言通过	0
Task 3 通用QEMU适配运行	按运行器实际输出断言通过	0
Task 4 通用QEMU适配运行	按运行器实际输出断言通过	0
Task4六个课程任务组合	对应行输出、sleep和timer均按运行器检查通过	0
Task4仅probe1：全体用户任务睡眠后的唤醒	运行器检查probe1的PASS及睡眠时间下界通过	0

临时地址重定位副本 + 独立测试BIOS的通用QEMU；测试实际执行内核切换、异常、系统调用、锁和定时器代码。不是课程QEMU/U-Boot或板端验收。

acore.py test和单probe运行器退出均为0；客体内核持续运行，运行器到观察时限后检查实际输出，超时本身不算通过。

> **从两个PASS还能读出什么**
>
> 阶段4的probe1/probe2先后start、在双方无系统调用的CPU循环结束前都已开始，然后检查用户寄存器、ENOSYS/EFAULT、锁与sleep并各自PASS。单probe另覆盖所有用户任务睡眠后的idle路径。它们比“屏幕上有输出”覆盖更多关键失误。

真正到lab时怎样分阶段观察

在解压的Project2-Acore根目录运行，使用课程现有编译器/QEMU/U-Boot配置。以下是课程环境预期命令，不代表本云环境已经直接运行了课程专用QEMU。

```
make run LAB_STAGE=1
make run LAB_STAGE=2
make run LAB_STAGE=3
make run LAB_STAGE=4
make run LAB_STAGE=4 TASKS='probe1 probe2'
make run LAB_STAGE=4 TASKS='probe1'
```

阶段	应该看到的现象
1	[COOP 1]、[COOP 2]计数都更新，主动让出后从原位置继续
2	两个mutex任务轮流出现acquired/release；持锁yield时另一个任务等待
3	sleep暂停后继续计数；coop_timer超过2秒；此阶段没有定时抢占
4默认	print1/2、lock1/2、sleep、timer各行持续变化，不依赖打印任务主动yield
4双probe	两个start、计算结束标志、锁交接与两个PASS
4单probe	唯一用户任务睡眠后恢复并出现PASS，验证idle及唤醒

这些任务往往通过VT100光标定位更新同一行，并不总是向下追加日志。观察画面时给终端足够行数；机器检查使用现有运行器重建屏幕，而不是把“运行了几秒没退出”当作通过。

把整条链自己讲一遍

1. TASKS决定映像里有哪些程序；boot传任务记录；内核把每个程序变成PCB、两块栈和两个初始帧。

2. 就绪队列选中任务，switch_to换成其内核帧，首次ret进入ret_from_exception，sret进入crt0，再call用户main。
3. 主动yield或阻塞请求先进入内核；调度器根据状态决定是否回队尾，并保存当前调用链。
4. 锁与sleep把暂不可运行的任务挂到等待队列；唤醒只让其READY，以后调度回来从原来的do_block之后继续。
5. Task4的时钟不需要用户同意：入口保存完整现场，处理器重设下次时钟并调度，最终恢复被打断的任务。

不属于本书已证明的能力

- Task5接口、动态创建/退出/kill、退出时自动释放锁。
- 页表带来的完整地址空间隔离、SMP多核同步、任意FPU程序上下文。
- 课程QEMU/U-Boot真实磁盘路径、loadbootd/DASIC及Pynq硬件验收。